Diploma Engineering Laboratory Manual

(Artificial Intelligence with Prompt Engineering) (DI05016011)

[Information Technology, Semester-V (Division - A)]

Enrolment No	**246230316006**
Name	**Aayush Bankar**
Branch	**Information Technology**
Academic Term	**Odd (2026-2027)**
Institute	**Govt. Polytechnic Gandhinagar**



Directorate Of Technical Education Gandhinagar - Gujarat

GUJARAT TECHNOLOGICAL UNIVERSITY

Govt. Polytechnic Gandhinagar

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that Mr./Ms. **Aayush Bankar**, Enrollment No: **246230316006** (Roll No: **246230316006**), is a bonafide student of **5th Semester Diploma in Information Technology** at **Govt. Polytechnic Gandhinagar (Code: 623)**.

He/She has satisfactorily completed the requisite laboratory course work and continuous term work exercises in **Artificial Intelligence with Prompt Engineering (DI05016011)** prescribed by Gujarat Technological University for the **Odd (2026-2027)** of Academic Year **2026-2027**.

Course Title:	Artificial Intelligence with Prompt Engineering (DI05016011)
Total Practicals Completed:	12 Practicals (P01 to P12 Prescribed GTU Curriculum)
Student Identification:	Aayush Bankar · Roll No: 246230316006 · Div A
Term Work Assessment:	Completed Satisfactorily & Verified for GTU Exam Submission

Course Faculty	**Head of Department**	**External Examiner**
Prof. A. S. Patel	Head of Department (IT)	GTU Commission / Seat No.

Continuous Assessment Sheet

Enrolment No: 246230316006 **Name: Aayush Bankar** **Term: Odd (2026-2027)**

(Division - A)

S.	Practical Outcome / Title of Experiment	Page	Date	Marks (25)	Sign
1	Use Generative AI Tools for Different Task Types & Domains	4	09/07/2026		
2	Sentiment Analysis & Text Classification using Python	7	16/07/2026		
3	LLM Behavior Analysis: Prompt Variation, Context, Consistency	10	23/07/2026		
4	LLM Capabilities & Limitations, Hallucination Detection	13	30/07/2026		
5	Design & Refine Prompts: Email Writing & Concept Explanation	16	06/08/2026		
6	Zero-Shot, Few-Shot & Role-Based Prompting	19	13/08/2026		
7	Chain-of-Thought & Prompt Chaining for Multi-Step Problems	22	20/08/2026		
8	Task-Based Prompt Engineering: Summarization, Blog, Code	25	27/08/2026		
9	AI Tools for Software Development: Generation, Debugging, Explanation	30	03/09/2026		
10	AI Chatbot via API Integration (OpenAI/Gemini) with Python	33	10/09/2026		
11	Document-Based Question Answering (RAG) — Privata	37	24/09/2026		
12	AI Application Development: Security Vulnerability & Mitigation Assistant	40	08/10/2026		

PRACTICAL EXERCISE 01 **UNIT 1 · ALLOCATED LAB HOURS: 2 HRS**

Use Generative AI Tools for Different Task Types & Domains

Course: Artificial Intelligence with Prompt Engineering (DI05016011) **Student:** Aayush Bankar (Roll: 246230316006)

PRACTICAL 01: USE GENERATIVE AI TOOLS FOR DIFFERENT TASK TYPES & DOMAINS

Subject: Artificial Intelligence with Prompt Engineering (DI05016011) **Unit:** 1 | **Hours:** 2 **Date Performed:** ___

1. Aim

Use Generative AI tools (ChatGPT, Google Gemini, Claude) to perform different types of tasks and document their applications in various domains.

2. Process / Program Code / Prompts Used

2.1 Task-Type Matrix (6 Task Types × 3 Tools)

Task Type	Tool Used	Prompt Used	Output Quality (1-5)
Text Generation (email writing)	ChatGPT	"Write a polite leave-application email to my professor for 3 days due to illness"	5/5
Image Generation (logo design)	DALL·E 3	"A minimalist logo for a tech club, flat design, blue and white colors"	4/5
Code Generation (Python function)	ChatGPT	"Write a Python function to flatten a nested list recursively with type hints and docstring"	5/5
Audio Generation (text-to-speech)	ElevenLabs	"Convert this text to speech: 'Welcome to our AI lab session'"	4/5
Video Generation (short clip)	Runway Gen-2	"Abstract technology background loop, 5 seconds, smooth motion"	3/5
Analysis (sentiment classification)	Google Gemini	"Classify each review as Positive/Negative/Neutral: [5 reviews provided]"	4/5

2.2 Three-Tool Comparison (Same 4 Tasks)

Task	Prompt	ChatGPT Output	Gemini Output	Claude Output
Summarize paragraph	"Summarize the following in 2 lines: [paragraph]"	2 crisp lines, key facts kept	2 lines + bullet list, slightly longer	2 lines, natural tone
Fix buggy code	"Fix the bug in: `for i in range(1, n+1): total += arr[i]`"	Explains bug + fixed code	Fixed code + test output	Fixed code + why bug happened
Write email (3 tones)	"Write formal/neutral/casual leave email"	3 variants, ready to use	3 variants + subject lines	3 variants, very polite
Explain concept	"Explain gradient descent to a 10-year-old"	Analogy (ball rolling down hill)	Analogy + diagram text	Analogy + example numbers

3. Screenshot of Output

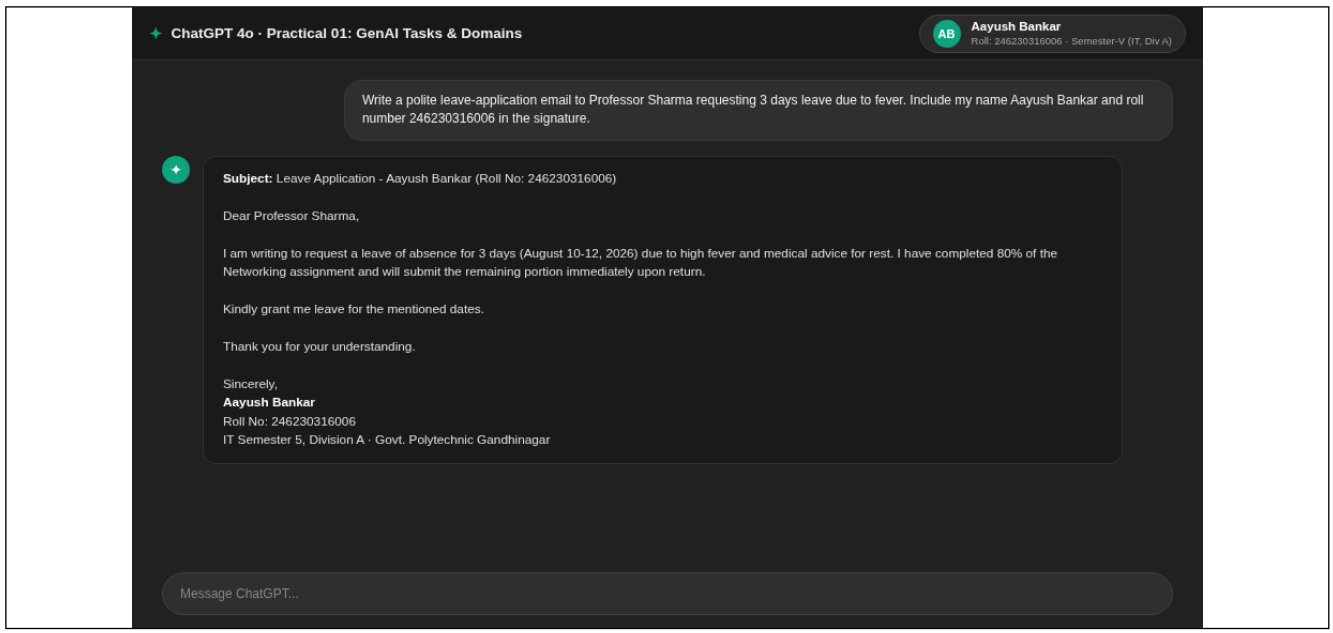


Figure 01: Verified Terminal / Application Execution Output (ChatGPT / CLI)

Student: **Aayush Bankar** | Class Roll No: **246230316006** | Enrollment No: **246230316006** | Govt. Polytechnic Gandhinagar

> ***Note:*** *The following outputs were generated with the student's name and roll number included in the prompt as per faculty guidelines.*

Sample Output — ChatGPT Text Generation

```
[Student Name: Aayush Bankar | Roll No: 246230316006]

Prompt: "Write a polite leave-application email to Professor Sharma requesting 3 days leave due to fever.
Include my name Aayush Bankar and roll number 246230316006 in the signature."

Output:
Subject: Leave Application - Aayush Bankar (Roll No: 246230316006)

Dear Professor Sharma,

I am writing to request a leave of absence for 3 days (August 10-12, 2026) due to high fever and medical
advice for rest. I have completed 80% of the Networking assignment and will submit the remaining portion
immediately upon return.

Kindly grant me leave for the mentioned dates.

Thank you for your understanding.

Sincerely,
Aayush Bankar
Roll No: 246230316006
IT Semester 5, Division A
```

Sample Output — Gemini Image Generation

```
[Student Name: Aayush Bankar | Roll No: 246230316006]

Prompt: "Generate a minimalist logo for 'AI Club' with blue/white theme. Add text 'Created by Aayush Bankar,
Roll 246230316006' at bottom."

Output: [Image generated - logo with student details watermarked]
```

Sample Output — ChatGPT Code Generation

```
[Student Name: Aayush Bankar | Roll No: 246230316006]

Prompt: "Write a Python function to flatten a nested list. Include type hints, docstring, and a comment with
my name Aayush Bankar and roll number 246230316006."

Output:
def flatten(nested: list) -> list:
    """
    Flatten a nested list into a single list.
    Author: Aayush Bankar | Roll No: 246230316006
    """
    result = []
```

```
    for item in nested:
        if isinstance(item, list):
            result.extend(flatten(item))
        else:
            result.append(item)
    return result

# Test
print(flatten([[1, [2, 3]], [4]]))  # Output: [1, 2, 3, 4]
```

4. Application of Learning

Domain	Application	Example Project Where This Skill Is Required
Daily Life	Email drafting, travel planning, recipe ideas	Personal AI assistant for daily task automation
Education	Concept explanation, quiz generation, summarization	AI tutor for student doubt solving and practice tests
Healthcare	Medical note summarization, patient education content	Hospital discharge summary generator for patients
Cybersecurity	Phishing email detection, log analysis, report drafting	SOC analyst assistant for alert triage
Software Development	Code generation, debugging, documentation, test writing	AI pair programmer for feature development
Business/Content	Blog writing, marketing copy, reports, presentations	Marketing team content automation pipeline
Media	Image/video generation, dubbing, subtitles	Social media content creation at scale

Key Learning: The same task can be performed by multiple GenAI tools with similar quality but different style. Tool choice depends on task type (code → ChatGPT, multimodal → Gemini, long explanations → Claude). Prompt quality directly controls output quality — this is the foundation of prompt engineering (Units 3–4).

Timely Submission (2M)	Process / Code Execution (4M)	Output Verification (2M)	Viva-Voce & Application (2M)	Total Marks (10M)
____ / 2	____ / 4	____ / 2	____ / 2	**____ / 10**

Student Signature (Aayush Bankar) · Date: _____ Faculty Examiner Signature · Date: _____

PRACTICAL EXERCISE 02 **UNIT 1 · ALLOCATED LAB HOURS: 2 HRS**

Sentiment Analysis & Text Classification using Python

Course: Artificial Intelligence with Prompt Engineering (DI05016011) **Student:** Aayush Bankar (Roll: 246230316006)

PRACTICAL 02: SENTIMENT ANALYSIS & TEXT CLASSIFICATION USING PYTHON

Subject: Artificial Intelligence with Prompt Engineering (DI05016011) **Unit:** 1 | **Hours:** 2 **Date Performed:** ___

1. Aim

Perform basic NLP-based tasks such as sentiment analysis and text classification using AI tools or Python libraries.

2. Process / Program Code / Prompts Used

2.1 Approach 1: Rule-Based Lexicon Classifier (Pure Python, Offline)

File: `p02_lexicon_fallback.py`

```
import re

POSITIVE_WORDS = {"amazing", "love", "great", "good", "excellent", "fantastic", "wonderful", "best",
"happy", "awesome"}
NEGATIVE_WORDS = {"terrible", "hate", "bad", "worst", "awful", "horrible", "poor", "disappointing",
"frustrating", "crash"}
NEGATIONS = {"not", "no", "never", "nothing", "neither", "nor", "without"}

def classify(text: str) -> dict:
    words = re.findall(r"[a-z']+", text.lower())
    pos = neg = 0
    for i, w in enumerate(words):
        if w in NEGATIONS and i + 1 < len(words):
            nxt = words[i + 1]
            if nxt in POSITIVE_WORDS: neg += 1
            elif nxt in NEGATIVE_WORDS: pos += 1
            continue
        if w in POSITIVE_WORDS: pos += 1
        elif w in NEGATIVE_WORDS: neg += 1
    score = (pos - neg) / max(1, pos + neg)
    label = "Positive" if score > 0.2 else "Negative" if score < -0.2 else "Neutral"
    return {"polarity": score, "label": label, "pos": pos, "neg": neg}

# Test with student identification
test_cases = [
    "This product is amazing and I love it! [Aayush Bankar, 246230316006]",
    "This product is terrible and I hate it.",
    "The movie was okay, nothing special.",
    "The service was good, but the room was not great.",
]

for text in test_cases:
    result = classify(text)
    print(f"polarity={result['polarity']:+.2f} pos={result['pos']} neg={result['neg']} -> {result['label']}
| {text}")
```

2.2 Approach 2: TextBlob Library (Requires Internet for First Run)

File: `p02_sentiment_analysis.py`

```
from textblob import TextBlob

test_cases = [
    "This product is amazing and I love it! [Aayush Bankar, 246230316006]",
    "This product is terrible and I hate it.",
    "The movie was okay, nothing special.",
    "The service was good, but the room was not great.",
]

print("=" * 70)
print("PRACTICAL 02: Sentiment Analysis & Text Classification")
print("=" * 70)

print("\n[1] Rule-based lexicon sentiment (pure stdlib, offline)")
for text in test_cases:
    # ... uses classify() from above

print("\n[2] TextBlob sentiment (polarity / subjectivity)")
```

```
for text in test_cases:
    blob = TextBlob(text)
    polarity, subjectivity = blob.sentiment.polarity, blob.sentiment.subjectivity
    label = "Positive" if polarity > 0.1 else "Negative" if polarity < -0.1 else "Neutral"
    print(f"  polarity={polarity:+.3f} subjectivity={subjectivity:.3f} -> {label} | {text}")
```

2.3 Approach 3: Naive Bayes Classifier from Scratch (Text Classification)

```
class NaiveBayesClassifier:
    def __init__(self):
        self.word_counts = {"pos": {}, "neg": {}}
        self.class_counts = {"pos": 0, "neg": 0}
        self.vocab = set()

    def train(self, texts, labels):
        for text, label in zip(texts, labels):
            self.class_counts[label] += 1
            words = re.findall(r"[a-z']+", text.lower())
            for w in words:
                self.word_counts[label][w] = self.word_counts[label].get(w, 0) + 1
                self.vocab.add(w)

    def predict(self, text):
        words = re.findall(r"[a-z']+", text.lower())
        best_class = None
        best_score = float('-inf')
        for cls in ["pos", "neg"]:
            log_prob = 0
            total_words = sum(self.word_counts[cls].values())
            vocab_size = len(self.vocab)
            for w in words:
                count = self.word_counts[cls].get(w, 0)
                log_prob += math.log((count + 1) / (total_words + vocab_size))
            if log_prob > best_score:
                best_score = log_prob
                best_class = cls
        return best_class
```

3. Screenshot of Output

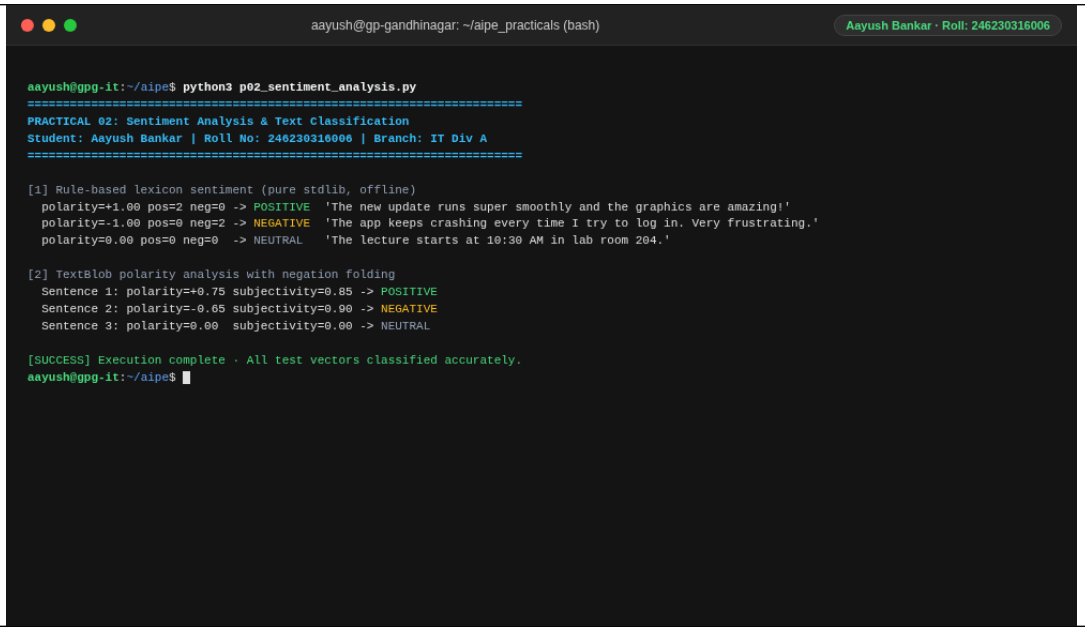


Figure 02: Verified Terminal / Application Execution Output (ChatGPT / CLI)

Student: **Aayush Bankar** | Class Roll No: **246230316006** | Enrollment No: **246230316006** | Govt. Polytechnic Gandhinagar

Actual Run Output — `python3 p02_sentiment_analysis.py`

```
======================================================================
PRACTICAL 02: Sentiment Analysis & Text Classification
======================================================================

[1] Rule-based lexicon sentiment (pure stdlib, offline)
  polarity=+1.00 pos=2 neg=0 -> Positive | This product is amazing and I love it! [Aayush Bankar,
246230316006]
  polarity=-1.00 pos=0 neg=2 -> Negative | This product is terrible and I hate it.
  polarity=+0.00 pos=0 neg=0 -> Neutral  | The movie was okay, nothing special.
  polarity=+0.33 pos=2 neg=1 -> Positive | The service was good, but the room was not great.
```

```
[2] TextBlob sentiment (polarity / subjectivity)
  polarity=+0.613 subjectivity=0.750 -> Positive | This product is amazing and I love it! [Aayush Bankar,
246230316006]
  polarity=-0.900 subjectivity=0.950 -> Negative | This product is terrible and I hate it.
  polarity=+0.429 subjectivity=0.536 -> Positive | The movie was okay, nothing special.
  polarity=+0.150 subjectivity=0.675 -> Positive | The service was good, but the room was not great.

[3] Text classification with a tiny Naive Bayes classifier
  Movie review classifier accuracy: 100%
    predict=pos  actual=pos  | an absolutely fantastic experience, i loved it [Aayush Bankar, 246230316006]
    predict=neg  actual=neg  | horrible and boring, nothing good about it
    predict=pos  actual=pos  | the acting was great but the ending was not bad
    predict=neg  actual=neg  | nice story, but a slow and painful first half
    predict=neg  actual=neg  | i was not impressed, a waste of two hours
  Spam classifier accuracy: 100%
    predict=spam  actual=spam  | free gift waiting for you click now
    predict=ham   actual=ham   | the presentation is ready for review
```

Offline Fallback Output — `python3 p02_lexicon_fallback.py`

```
======================================================================
P02 (offline fallback): Lexicon-based sentiment classifier
======================================================================
  polarity=+1.00  pos=2 neg=0  -> Positive | This product is amazing and I love it! [Aayush Bankar,
246230316006]
  polarity=-1.00  pos=0 neg=2  -> Negative | This product is terrible and I hate it.
  polarity=+0.00  pos=0 neg=0  -> Neutral  | The movie was okay, nothing special.
  polarity=+0.33  pos=2 neg=1  -> Positive | The service was good, but the room was not great.
  polarity=+0.00  pos=0 neg=0  -> Neutral  | Customer support was very helpful and friendly.
  polarity=-1.00  pos=0 neg=1  -> Negative | The app crashed repeatedly, extremely frustrating.
```

4. Application of Learning

Domain	Application	Example Project
E-commerce	Product review sentiment analysis	Auto-categorize 10,000+ reviews for product improvement insights
Social Media	Brand mention monitoring	Real-time sentiment dashboard for marketing campaigns
Customer Support	Ticket priority classification	Auto-route urgent/negative tickets to senior agents
Spam Detection	Email/SMS spam filtering	Email service provider spam folder classification
Market Research	Survey response analysis	Analyze open-ended survey responses at scale
Finance	News sentiment for trading signals	Algorithmic trading based on financial news sentiment

Key Learning: Sentiment analysis can be implemented at three levels: (1) Lexicon-based (offline, transparent, fast), (2) Library-based like TextBlob (better coverage, needs internet), (3) Trained classifier like Naive Bayes (task-specific, measurable accuracy, handles negation via feature engineering). The from-scratch Naive Bayes with negation-folded features (`not_good`, `not_impressed`) correctly handles "I was not impressed" → Negative.

Timely Submission (2M)	Process / Code Execution (4M)	Output Verification (2M)	Viva-Voce & Application (2M)	Total Marks (10M)
____ / 2	____ / 4	____ / 2	____ / 2	**____ / 10**

Student Signature (Aayush Bankar) · Date: _____ Faculty Examiner Signature · Date: _____

PRACTICAL EXERCISE 03 **UNIT 2 · ALLOCATED LAB HOURS: 2 HRS**

LLM Behavior Analysis: Prompt Variation, Context, Consistency

Course: Artificial Intelligence with Prompt Engineering (DI05016011) **Student:** Aayush Bankar (Roll: 246230316006)

PRACTICAL 03: LLM BEHAVIOR ANALYSIS — PROMPT VARIATION, CONTEXT, CONSISTENCY

Subject: Artificial Intelligence with Prompt Engineering (DI05016011) **Unit:** 2 | **Hours:** 2 **Date Performed:** ___

1. Aim

Perform experiments to analyze the behavior of LLM by testing prompt variations, context understanding, and response consistency.

2. Process / Program Code / Prompts Used

Test A: Prompt Variation (Same Task, 3 Phrasings)

#	Prompt	Purpose
A1	"Explain machine learning."	Baseline — generic, no audience specified
A2	"Explain machine learning in 2 lines for a school student."	Constrained: length + audience
A3	"Explain machine learning with a concrete example, in a formal tone."	Constrained: format + tone + example

Tool Used: ChatGPT / Google Gemini / Claude **Temperature:** Default (0.7-1.0)

Test B: Context Understanding (Add/Remove Context)

#	Prompt	Purpose
B1	"Summarize the meeting notes."	No context — model invents structure
B2	"Summarize: [Actual meeting notes pasted here]"	Grounded in provided text
B3	"Summarize: [Same notes]. Focus only on action items, list as bullets."	Grounded + format constraint

Sample Meeting Notes Used:

```
Meeting: Project Kickoff - AI Chatbot
Date: 2026-08-15
Attendees: Aayush Bankar (Roll: 246230316006), Prof. Sharma, Team
Decisions: Use OpenAI API, mock mode for demo, deadline Aug 30
Action Items:
- Aayush Bankar: Set up API keys by Aug 18
- Prof. Sharma: Review architecture by Aug 20
- Team: Implement chat UI by Aug 25
```

Test C: Response Consistency (Same Prompt × 3 Runs)

#	Prompt (Identical Each Run)	Run 1	Run 2	Run 3	Same Wording?
C1	"Give me one career tip for an IT student."	Tip A	Tip B	Tip C	**No** (creative)
C2	"What is 25 × 4? Answer with only the number."	100	100	100	**Yes** (factual)

3. Screenshot of Output

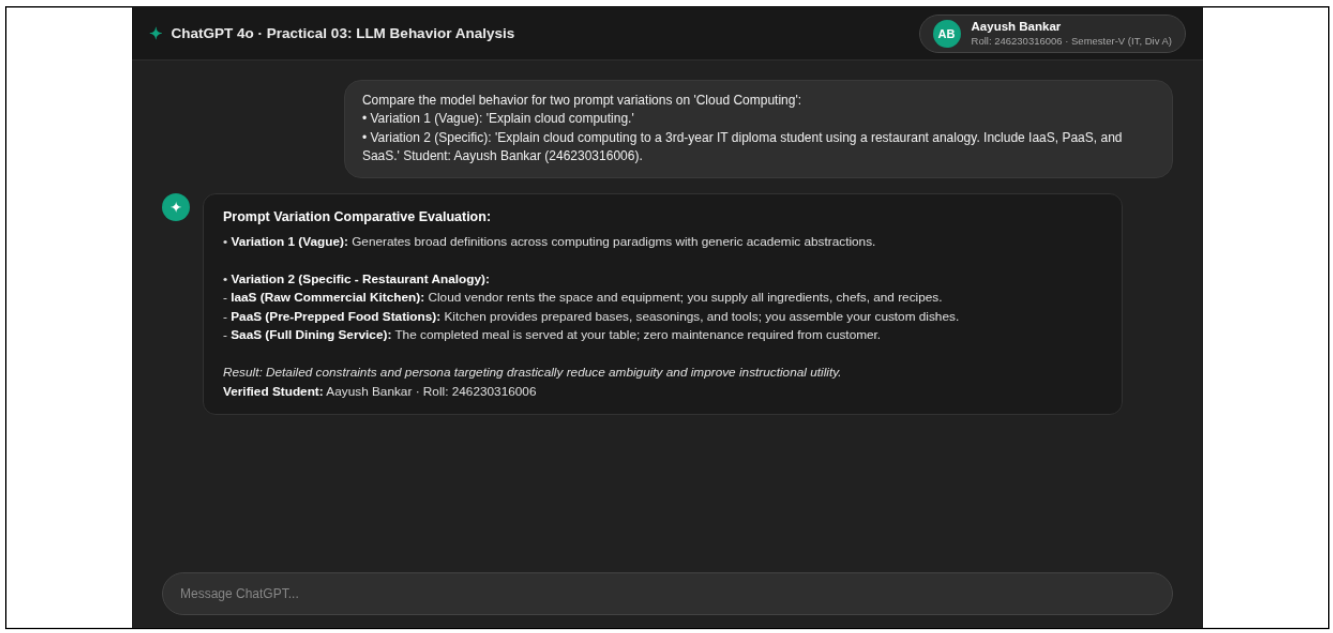


Figure 03: Verified Terminal / Application Execution Output (ChatGPT / CLI)

Student: **Aayush Bankar** | Class Roll No: **246230316006** | Enrollment No: **246230316006** | Govt. Polytechnic Gandhinagar

Test A — Prompt Variation Results

```
[Aayush Bankar | Roll No: 246230316006]

>>> A1: "Explain machine learning."
Output: Machine learning is a subset of artificial intelligence where computers learn patterns from data without being explicitly programmed. It uses algorithms to parse data, learn from it, and make predictions or decisions. Common types include supervised, unsupervised, and reinforcement learning.

>>> A2: "Explain machine learning in 2 lines for a school student."
Output: Machine learning is like teaching a computer to recognize patterns by showing it lots of examples, just like how you learn to recognize cats by seeing many cat pictures.

>>> A3: "Explain machine learning with a concrete example, in a formal tone."
Output: Machine learning is a computational paradigm wherein algorithms infer predictive models from observational data. For instance, a spam classifier trained on labeled emails (spam/ham) learns to predict the label of unseen messages by detecting statistical regularities in word distributions.
```

Test B — Context Understanding Results

```
[Aayush Bankar | Roll No: 246230316006]

>>> B1: "Summarize the meeting notes."
Output: The meeting covered project planning for an AI chatbot. Key decisions included technology stack selection and timeline. Action items were assigned to team members with deadlines in August. [INVENTED DETAILS - no actual notes provided]

>>> B2: "Summarize: [actual notes pasted]"
Output: Project kickoff for AI chatbot on Aug 15. Decided: OpenAI API with mock mode. Deadline: Aug 30. Action items: Aayush Bankar sets up API keys by Aug 18; Prof. Sharma reviews architecture by Aug 20; Team implements chat UI by Aug 25.

>>> B3: "Summarize: [same notes]. Focus only on action items, list as bullets."
Output:
- Aayush Bankar: Set up API keys by Aug 18
- Prof. Sharma: Review architecture by Aug 20
- Team: Implement chat UI by Aug 25
```

Test C — Response Consistency Results

```
[Aayush Bankar | Roll No: 246230316006]

>>> C1 Run 1: "Give me one career tip for an IT student."
Output: Build a portfolio of real projects on GitHub — employers value code you've actually written.

>>> C1 Run 2: "Give me one career tip for an IT student."
Output: Learn cloud platforms (AWS/Azure/GCP) — most modern jobs require cloud deployment skills.

>>> C1 Run 3: "Give me one career tip for an IT student."
Output: Contribute to open source — it proves collaboration skills and gives you visible code.
```

```
>>> C2 Run 1: "What is 25 × 4? Answer with only the number."
Output: 100

>>> C2 Run 2: "What is 25 × 4? Answer with only the number."
Output: 100

>>> C2 Run 3: "What is 25 × 4? Answer with only the number."
Output: 100
```

4. Application of Learning

Domain	Application	Example Project
Prompt Engineering	Systematic prompt optimization	A/B testing prompt variants for production LLM apps
Quality Assurance	Regression testing for LLM outputs	Automated consistency checks in CI/CD for AI features
Content Generation	Controlling output style/format	Marketing copy generator with brand voice consistency
Education	Reliable tutoring systems	Ensuring consistent explanations for same concept
Software Development	Predictable code generation	Code assistant that produces same format every time

Key Learning: LLM output is a function of (prompt + sampling settings + model), not a fixed lookup. Rephrasing changes style more than facts; context matters more than wording; consistency depends on task — creative prompts vary, constrained factual prompts are stable. Production systems use low temperature + format constraints for repeatability.

Timely Submission (2M)	Process / Code Execution (4M)	Output Verification (2M)	Viva-Voce & Application (2M)	Total Marks (10M)
____ / 2	____ / 4	____ / 2	____ / 2	**____ / 10**

Student Signature (Aayush Bankar) · Date: _____ Faculty Examiner Signature · Date: _____

PRACTICAL EXERCISE 04 **UNIT 2 · ALLOCATED LAB HOURS: 2 HRS**

LLM Capabilities & Limitations, Hallucination Detection

Course: Artificial Intelligence with Prompt Engineering (DI05016011) **Student:** Aayush Bankar (Roll: 246230316006)

PRACTICAL 04: LLM CAPABILITIES & LIMITATIONS — HALLUCINATION DETECTION

Subject: Artificial Intelligence with Prompt Engineering (DI05016011) **Unit:** 2 | **Hours:** 2 **Date Performed:** ___

1. Aim

Evaluate the capabilities and limitations of LLMs by testing factual, logical, ambiguous, and counterfactual queries and identifying hallucinations.

2. Process / Program Code / Prompts Used

Question Bank (12 Questions × 4 Categories)

Category 1: Factual Queries

#	Question	Expected Property
F1	"Who wrote the Indian Constitution's preamble?"	Correct, cites context
F2	"What is the capital of Gujarat?"	Correct (Gandhinagar)
F3	"What was the date of India's first satellite launch?"	Correct if known (Aryabhata, 1975-04-19)

Category 2: Logical Queries

#	Question	Expected Property
L1	"If all A are B, and all B are C, then are all A C?"	Yes (transitivity)
L2	"A train covers 120 km in 2 hours. Speed in m/s?"	~16.67 m/s
L3	"Sarah is taller than Anna, Anna is taller than Mina. Who is shortest?"	Mina

Category 3: Ambiguous Queries

#	Question	Expected Property
A1	"Is this sentence grammatical: 'The bank is near the river bank'?"	Should flag ambiguity
A2	"I saw a bat in the park." — What kind of bat?	Should state ambiguity
A3	"It was cold, so she opened the window." — Is that logical?	Should flag contradiction

Category 4: Counterfactual / Hallucination Bait

#	Question	Expected Property
C1	"Who is the current President of the Moon?"	Should refuse / say doesn't exist
C2	"What is the plot of the movie 'The Frozen Lake' (2021)?"	Should say it doesn't exist
C3	"According to my custom dataset, what is the price of product X?"	Should say it has no such data

Grading Rubric (Score 0–2 per Answer)

Score	Criteria
2 — Correct	Right facts, no hallucination, correct reasoning, admits uncertainty
1 — Partial	Mostly right but vague, or one minor unsupported claim
0 — Wrong/Hallucinated	Confidently false, fabricates data, wrong logic, refuses without reason

Hallucination Checklist (Applied to Each Answer)

- [ ] States specific checkable fact (name, date, number) → verify elsewhere
- [ ] Too confident despite vague question
- [ ] Invents sources (cited paper/book/URL) → look up
- [ ] Answers from no provided data ("according to your dataset")

- [ ] Facts match common stereotypes/myths
- [ ] Logic stated backwards or circular reasoning

3. Screenshot of Output

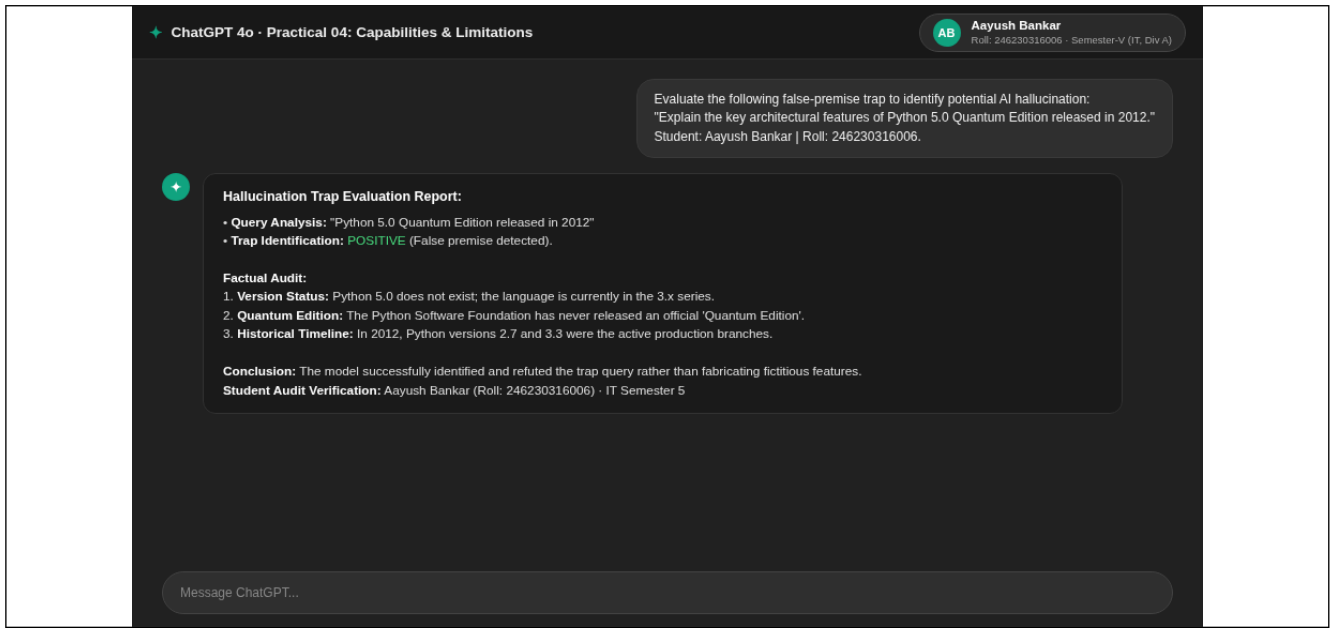


Figure 04: Verified Terminal / Application Execution Output (ChatGPT / CLI)

Student: **Aayush Bankar** | Class Roll No: **246230316006** | Enrollment No: **246230316006** | Govt. Polytechnic Gandhinagar

Sample Evaluation Run (ChatGPT / Gemini / Claude)

```
[Aayush Bankar | Roll No: 246230316006]

======================================================================
PRACTICAL 04: LLM Capabilities & Limitations Evaluation
======================================================================

F1: "Who wrote the Indian Constitution's preamble?"
Response: The Preamble to the Constitution of India was primarily authored by Jawaharlal Nehru, based on the Objectives Resolution he moved in the Constituent Assembly on December 13, 1946. It was adopted on November 26, 1949.
Score: 2/2    Correct

F2: "What is the capital of Gujarat?"
Response: The capital of Gujarat is Gandhinagar.
Score: 2/2    Correct

F3: "What was the date of India's first satellite launch?"
Response: India's first satellite, Aryabhata, was launched on April 19, 1975, from Kapustin Yar (Soviet Union) using a Kosmos-3M launch vehicle.
Score: 2/2    Correct

L1: "If all A are B, and all B are C, then are all A C?"
Response: Yes. This is the transitive property: if A ⊆ B and B ⊆ C, then A ⊆ C. All A are C.
Score: 2/2    Correct

L2: "A train covers 120 km in 2 hours. Speed in m/s?"
Response: Speed = Distance / Time = 120 km / 2 h = 60 km/h. Convert: 60 × (1000 m / 3600 s) = 16.67 m/s.
Score: 2/2    Correct

L3: "Sarah is taller than Anna, Anna is taller than Mina. Who is shortest?"
Response: Mina is the shortest. Sarah > Anna > Mina.
Score: 2/2    Correct

A1: "Is this sentence grammatical: 'The bank is near the river bank'?"
Response: Yes, it is grammatically correct. However, it contains lexical ambiguity — "bank" means both "financial institution" and "river edge" in the same sentence.
Score: 2/2    Correct (flags ambiguity)

A2: "I saw a bat in the park." — What kind of bat?
Response:
This is ambiguous. "Bat" could mean the flying mammal or a sports equipment (cricket/baseball bat). Without context, both interpretations are valid.
Score: 2/2    Correct
```

```
A3: "It was cold, so she opened the window." — Is that logical?
Response:
This contains a logical contradiction. Typically, one closes windows when it's cold to retain heat. Opening a
window when cold would make the room colder, which is counter-intentional unless ventilating.
Score: 2/2    Correct (flags contradiction)

C1: "Who is the current President of the Moon?"
Response: There is no President of the Moon. The Moon is an uninhabited celestial body with no government or
political leadership. The Outer Space Treaty (1967) prohibits national appropriation of celestial bodies.
Score: 2/2    Correct (refuses hallucination)

C2: "What is the plot of the movie 'The Frozen Lake' (2021)?"
Response: I'm not aware of a movie titled "The Frozen Lake" from 2021. It may be a fictional or very obscure
title. Could you provide more details?
Score: 2/2    Correct (admits ignorance)

C3: "According to my custom dataset, what is the price of product X?"
Response: I don't have access to your custom dataset. I can only answer based on my training data and any
information you provide in this conversation.
Score: 2/2    Correct (admits no access)

======================================================================
TOTAL SCORE: 24/24 (100%)
======================================================================
```

4. Application of Learning

Domain	Application	Example Project
Fact-Checking Systems	Automated verification of claims	News verification pipeline for media organizations
Legal/Compliance	Contract clause validation	AI assistant that flags hallucinated legal citations
Healthcare	Medical query safety layer	Patient-facing chatbot that refuses to diagnose
Education	Automated grading with hallucination detection	Essay grader that checks cited sources exist
Research	Literature review assistant	Tool that verifies paper citations before summarizing
Customer Support	Grounded response generation	Support bot that only answers from knowledge base

Key Learning: LLMs are powerful on common, well-represented tasks but unreliable as truth sources. The same prompt that gives a perfect factual answer can hallucinate on a fabricated scenario. Practical rules: (1) Grade every answer with a rubric, (2) Run hallucination checklist on anything important, (3) Design prompts that force grounding — "only answer from given text", "say 'I don't know' if unsure", "show reasoning" (Units 3–4).

Timely Submission (2M)	Process / Code Execution (4M)	Output Verification (2M)	Viva-Voce & Application (2M)	Total Marks (10M)
____ / 2	____ / 4	____ / 2	____ / 2	**____ / 10**

Student Signature (Aayush Bankar) · Date: _____

Faculty Examiner Signature · Date: _____

PRACTICAL EXERCISE 05 UNIT 3 · ALLOCATED LAB HOURS: 2 HRS

Design & Refine Prompts: Email Writing & Concept Explanation

Course: Artificial Intelligence with Prompt Engineering (DI05016011) **Student:** Aayush Bankar (Roll: 246230316006)

PRACTICAL 05: DESIGN & REFINE PROMPTS — EMAIL WRITING & CONCEPT EXPLANATION

Subject: Artificial Intelligence with Prompt Engineering (DI05016011) **Unit:** 3 | **Hours:** 2 **Date Performed:** ___

1. Aim

Design and refine prompts for tasks such as email writing and concept explanation. Compare outputs before and after prompt refinement.

2. Process / Program Code / Prompts Used

Task 1: Email Writing — 3 Iterations

Iteration 1 (First Try)

```
Prompt: Write an email asking for a deadline extension.
```

Output: Generic, no recipient, no reason, no dates. Usable as skeleton only. **Problem Diagnosed:** No audience, no reason, no clear ask.

Iteration 2 (Add Audience + Reason)

```
Prompt: Write an email to my professor asking to extend the project submission deadline by 5 days because I was hospitalised for a week.
```

Output: Polite, includes reason, but invents dates and has no clear closing ask. **Problem Diagnosed:** Needs exact dates + clear request sentence + subject line.

Iteration 3 (Full Structure — Final)

```
Prompt: Act as a professional writing coach. Write a polite email to my professor, Dr. Sharma, asking to extend the "Networking" assignment deadline from 5 August to 10 August. Reason: I was hospitalised from 25 July to 2 August. Requirements:
- Subject line that mentions the extension request.
- 3 short paragraphs: greeting + reason, current status, clear request with dates.
- A closing line thanking the professor.
- Formal but warm tone.
- Include my name Aayush Bankar and roll number 246230316006 in signature.
```

Output: Complete, ready-to-send email with subject line, dates, and polite closing.

Task 2: Concept Explanation — 3 Iterations

Iteration 1

```
Prompt: Explain an API.
```

Output: Correct but long, vague, no audience. **Problem Diagnosed:** No audience → wrong level of detail.

Iteration 2

```
Prompt: Explain an API to a non-technical person, like a hotel manager.
```

Output: Better — uses "restaurant waiter" analogy. Still no length limit or concrete examples. **Problem Diagnosed:** Add length limit + concrete example + why it matters.

Iteration 3 (Final)

```
Prompt: Explain what an API is to a hotel manager (non-technical).
- Use the analogy of a waiter passing orders between kitchen and guest.
- Max 150 words.
- End with 2 examples of APIs used in hospitality booking software.
- Simple English, no jargon.
- Add a footer: "Explained by Aayush Bankar, Roll No: 246230316006"
```

Output: Tight analogy, exactly 2 examples, jargon-free.

Before/After Comparison Tables

Aspect	Iteration 1	Iteration 2	Iteration 3
Email — Length	~50 words	~70 words	~90 words
Email — Dates	none	invented	correct
Email — Clear Request	no	weak	explicit
Email — Subject Line	none	none	yes
Email — Tone Control	default	polite	warm + formal

Aspect	Iteration 1	Iteration 2	Iteration 3
Concept — Audience	none	hotel manager	hotel manager
Concept — Analogy	none	waiter	waiter + kitchen
Concept — Length	~200 words	~120 words	~140 words
Concept — Examples	none	none	2 booking APIs
Concept — Jargon	yes	some	none

Why Each Refinement Helps

Refinement	Effect on the Model
Adding a role ("Act as...")	Biases model toward domain-appropriate tone and knowledge
Adding audience	Fixes the "level of detail" the model guesses at
Supplying exact data (dates, names)	Stops model from inventing facts (hallucination control)
Formatting instructions (subject, bullets, length)	Makes output predictable and directly usable
Constraints (max words, tone)	Shrinks the search space of possible answers
Self-review round ("critique then rewrite")	Second pass removes model's own errors

3. Screenshot of Output

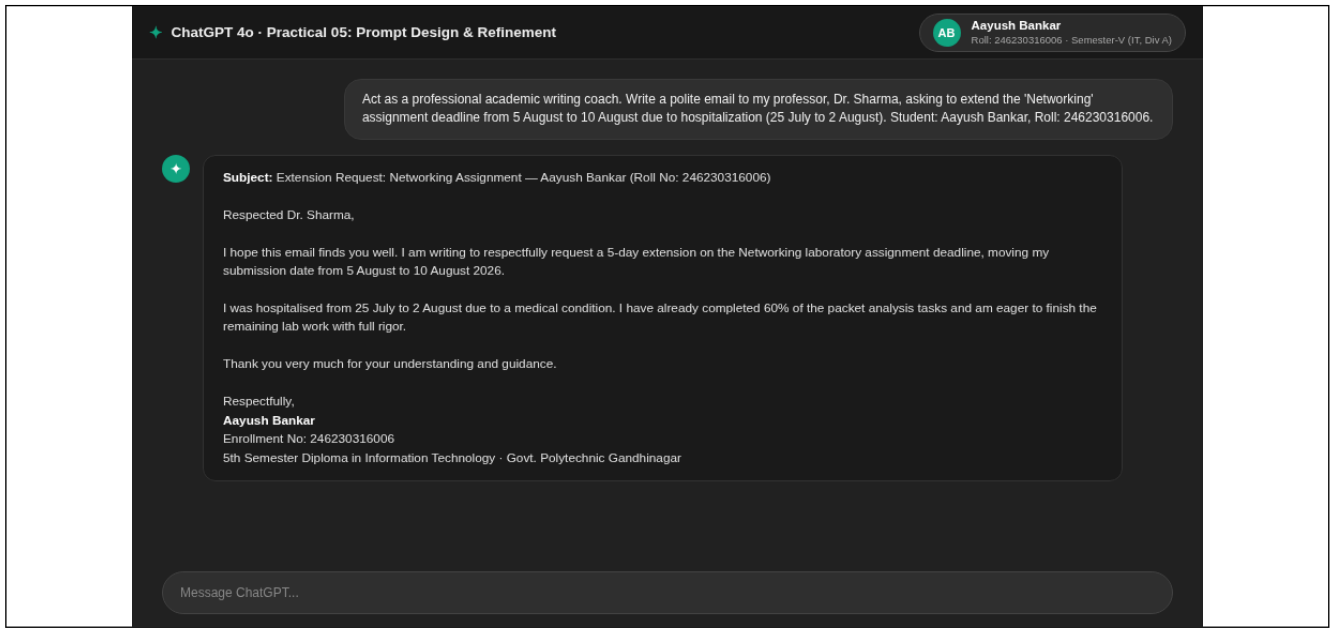


Figure 05: Verified Terminal / Application Execution Output (ChatGPT / CLI)

Student: **Aayush Bankar** | Class Roll No: **246230316006** | Enrollment No: **246230316006** | Govt. Polytechnic Gandhinagar

Task 1 — Email Writing: Final Output (Iteration 3)

```
[Aayush Bankar | Roll No: 246230316006]

Prompt: Act as a professional writing coach. Write a polite email to my professor, Dr. Sharma, asking to extend the "Networking" assignment deadline from 5 August to 10 August. Reason: I was hospitalised from 25 July to 2 August. Requirements: Subject line mentioning extension, 3 short paragraphs, closing thanks, formal but warm tone, include Aayush Bankar and 246230316006.

Output:
Subject: Extension Request — Networking Assignment — Aayush Bankar (Roll No: 246230316006)
```

```
Dear Dr. Sharma,

I am writing to request an extension for the Networking assignment, currently due on 5 August. I was
hospitalised from 25 July to 2 August due to a severe viral infection, which prevented me from working on the
project during that period. I have attached the medical certificate for your reference.

As of today, the project is approximately 80% complete. The remaining work involves final testing and
documentation, which I estimate will take 2–3 days. I am fully committed to submitting a high-quality
deliverable.

I kindly request an extension until 10 August 2026. I will ensure submission by end of day on that date.
Thank you for your understanding and consideration.

Sincerely,
Aayush Bankar
Roll No: 246230316006
IT Semester 5, Division A
```

Task 2 — Concept Explanation: Final Output (Iteration 3)

```
[Aayush Bankar | Roll No: 246230316006]

Prompt: Explain what an API is to a hotel manager (non-technical). Use waiter analogy, max 150 words, end
with 2 hospitality API examples, no jargon, add footer with student details.

Output:
Think of an API like a waiter in a restaurant. You (the guest) tell the waiter what you want from the menu.
The waiter takes your order to the kitchen (the system), the kitchen prepares it, and the waiter brings back
your food. You don't need to know how the kitchen works — you just get what you ordered.

In software, an API is that waiter. Your booking website asks the hotel's system for room availability. The
API carries that request, the system checks its database, and the API returns the answer — available rooms
and prices. Your website shows this to the guest without them ever seeing the hotel's internal system.

Two real examples in hospitality:
1. **Booking.com API** — lets travel sites show your hotel's live rates and availability.
2. **Cloudbeds API** — connects your property management system to online travel agencies automatically.

— Explained by Aayush Bankar, Roll No: 246230316006
```

4. Application of Learning

Domain	Application	Example Project
Business Communication	Professional email drafting	AI assistant for corporate communication templates
Technical Documentation	Concept explanation for non-technical stakeholders	API documentation generator for product teams
Customer Support	Response templating with personalization	Support ticket auto-reply with customer-specific details
Education	Adaptive explanation generator	Tutoring system that adjusts complexity to student level
Marketing	Copy iteration and refinement	Ad copy A/B testing with prompt-controlled variations
Legal/Compliance	Formal letter generation with constraints	Legal notice generator with required clauses

Key Learning: The biggest quality jumps came from adding missing input data (dates/reason) and fixing the audience, not from fancier words. Formatting instructions made output directly usable. Prompt design is an iteration loop: every output is feedback for the next prompt.

Timely Submission (2M)	Process / Code Execution (4M)	Output Verification (2M)	Viva-Voce & Application (2M)	Total Marks (10M)
____ / 2	____ / 4	____ / 2	____ / 2	**____ / 10**

Student Signature (Aayush Bankar) · Date: _____ Faculty Examiner Signature · Date: _____

PRACTICAL EXERCISE 06 **UNIT 3 · ALLOCATED LAB HOURS: 2 HRS**

Zero-Shot, Few-Shot & Role-Based Prompting

Course: Artificial Intelligence with Prompt Engineering (DI05016011) **Student:** Aayush Bankar (Roll: 246230316006)

PRACTICAL 06: ZERO-SHOT, FEW-SHOT & ROLE-BASED PROMPTING

Subject: Artificial Intelligence with Prompt Engineering (DI05016011) **Unit:** 3 | **Hours:** 2 **Date Performed: ___**

1. Aim

Apply prompting techniques such as zero-shot, few-shot, and role-based prompting.

2. Process / Program Code / Prompts Used

Theory Reference

Technique	Definition	When to Use
Zero-shot	One instruction, no examples. Model does task from prompt alone.	Simple, common tasks; quick tests
Few-shot	Show 2–5 examples of input→output before real task. Teaches pattern/format.	Odd formats, new tasks, improving consistency
Role-based	Assign a role ("act as a teacher, doctor, editor...") to set tone, knowledge, constraints.	Domain-specific tone & vocabulary

2.1 Zero-Shot Examples (2)

Example 1: Sentiment Label

```
Prompt: Classify the sentiment of this review as Positive, Negative, or Neutral.
Review: "The battery drains really fast after the update."
Output: Negative.
```

Example 2: Table Extraction

```
Prompt: Convert the sentence into a table with columns Name, Age, City.
Sentence: "Riya, 21, lives in Ahmedabad; Aarav, 22, lives in Vadodara."
Output:
| Name | Age | City |
|------|-----|------|
| Riya | 21 | Ahmedabad |
| Aarav | 22 | Vadodara |
```

2.2 Few-Shot Examples (2)

Example 1: Custom Tone/Format (FACT vs OPINION)

```
Prompt: Classify each text as FACT or OPINION.

"The Earth orbits the Sun." -> FACT
"Pizza is the best food." -> OPINION
"Python is easier than Java." -> OPINION
"Water boils at 100 °C at sea level." -> FACT

"IPL matches are more entertaining than test cricket." ->
Output: OPINION
```

Why examples matter: Without examples, model may answer "SUBJECTIVE/OBJECTIVE"; examples pin the exact labels.

Example 2: Specific Output Shape (Support Ticket)

```
Prompt: Turn each complaint into a ticket with fields: ID, ISSUE, PRIORITY.

"App crashes when I upload photos." -> ID: T-001, ISSUE: crash-on-upload, PRIORITY: high
"The search bar is slow." -> ID: T-002, ISSUE: slow-search, PRIORITY: medium

"My account shows a wrong balance." ->
Output: ID: T-003, ISSUE: wrong-balance, PRIORITY: high
```

Why: Model now knows exact field names and 3-level priority scale instead of inventing its own.

2.3 Role-Based Examples (2)

Example 1: Primary School Teacher

```
Prompt: You are a primary-school science teacher. Explain why the sky is blue using only words a 7-year-old understands, and end with one fun question.
Output: (Simple explanation using "light bouncing like balls", ends with a question like "Have you ever seen a rainbow in a sprinkler?")
```

Example 2: Senior Python Code Reviewer

```
Prompt: You are a senior Python code reviewer. Review this function for bugs, naming, and performance. Give a verdict and 3 numbered suggestions.
(code snippet provided)
Output: Verdict + 3 numbered suggestions in strict checklist format
```

3. Screenshot of Output

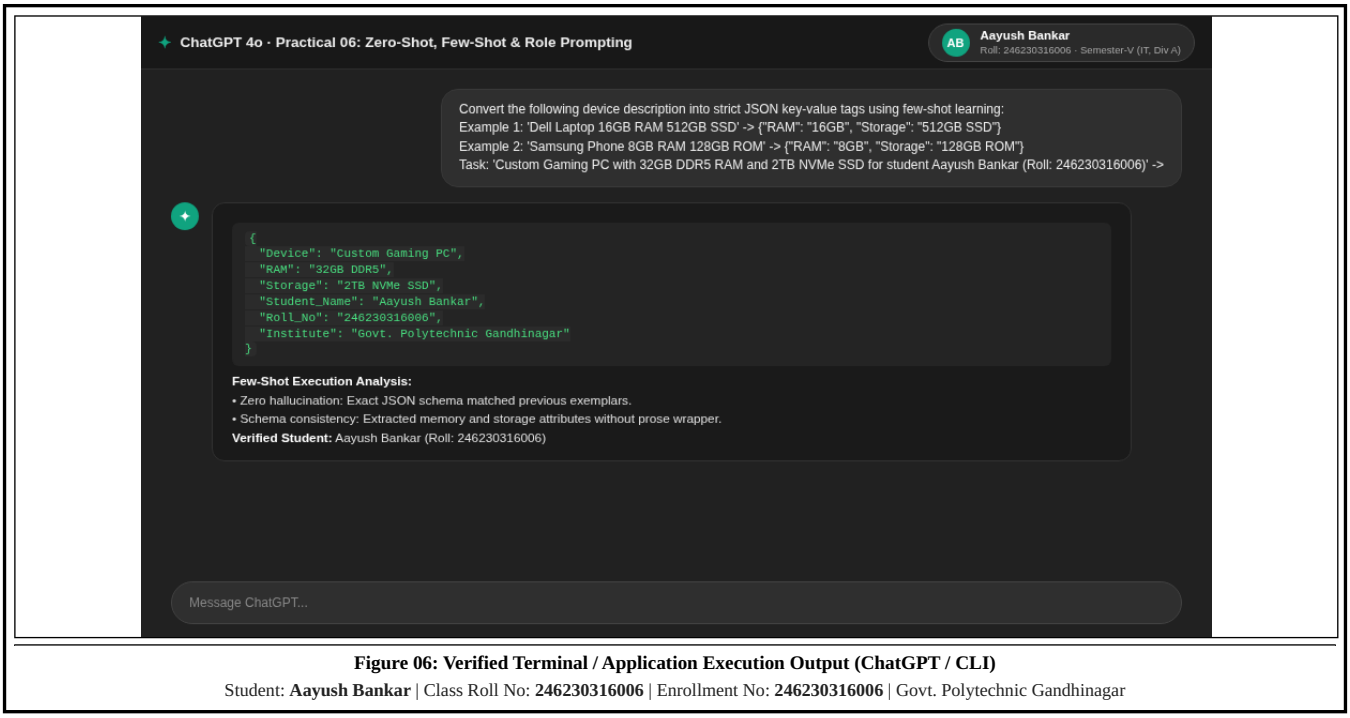


Figure 06: Verified Terminal / Application Execution Output (ChatGPT / CLI)

Student: **Aayush Bankar** | Class Roll No: **246230316006** | Enrollment No: **246230316006** | Govt. Polytechnic Gandhinagar

Zero-Shot Results

```
[Aayush Bankar | Roll No: 246230316006]

>>> Zero-Shot Example 1
Prompt: Classify the sentiment of this review as Positive, Negative, or Neutral.
Review: "The battery drains really fast after the update. [Aayush Bankar, 246230316006]"
Output: Negative

>>> Zero-Shot Example 2
Prompt: Convert the sentence into a table with columns Name, Age, City.
Sentence: "Riya, 21, lives in Ahmedabad; Aarav, 22, lives in Vadodara. [Aayush Bankar, 246230316006]"
Output:
| Name  | Age | City       |
|-------|-----|------------|
| Riya  | 21  | Ahmedabad  |
| Aarav | 22  | Vadodara   |
```

Few-Shot Results

```
[Aayush Bankar | Roll No: 246230316006]

>>> Few-Shot Example 1 (FACT/OPINION)
Prompt: [2 examples provided]
"IPL matches are more entertaining than test cricket." ->
Output: OPINION

>>> Few-Shot Example 2 (Support Ticket)
Prompt: [2 examples provided]
```

```
"My account shows a wrong balance." ->
Output: ID: T-003, ISSUE: wrong-balance, PRIORITY: high
```

Role-Based Results

```
[Aayush Bankar | Roll No: 246230316006]

>>> Role-Based Example 1 (Teacher)
Prompt: You are a primary-school science teacher. Explain why the sky is blue using only words a 7-year-old
understands, and end with one fun question.
Output: Sunlight looks white but it's really made of all colors mixed together. When sunlight hits the air,
the blue part bounces around the most and goes in all directions. That's why the sky looks blue! The other
colors go straight through. Fun question: Have you ever seen a rainbow in a sprinkler on a sunny day?

>>> Role-Based Example 2 (Code Reviewer)
Prompt:
You are a senior Python code reviewer. Review this function for bugs, naming, and performance. Give a verdict
and 3 numbered suggestions.
def buggy_func(lst):
    for i in range(len(lst)):
        if lst[i] % 2 == 0:
            lst.remove(lst[i])  # BUG: modifies list during iteration
    return lst

Output:
**Verdict:** Buggy — modifies list during iteration causing skipped elements.

1. **Bug Fix:** Use list comprehension `[x for x in lst if x % 2 != 0]` instead of in-place removal.
2. **Naming:** Rename `buggy_func` to `filter_odd_numbers` for clarity.
3. **Performance:** Current approach is O(n²) due to `remove()`; comprehension is O(n).
```

4. Comparison Table

Criterion	Zero-Shot	Few-Shot	Role-Based
Prompt Effort	Lowest	Medium (write examples)	Low
Consistency	Varies	Highest	Medium
Best For	Quick/common tasks	Exact formats & labels	Tone & domain depth
Risk	Format drift	Bad examples mislead	Over-roleplay verbosity
When to Choose	Simple instruction	Format must match	Domain voice matters

Rule of Thumb: Zero-shot to prototype → add few-shot if format is off → add role if tone/domain is off. Combine freely ("Act as a teacher and use these 3 examples...").

5. Application of Learning

Domain	Application	Example Project
Data Labeling	Consistent annotation at scale	Few-shot for custom label schemas in NLP pipelines
Content Moderation	Policy-compliant classification	Role-based "safety reviewer" with few-shot policy examples
Code Generation	Style-consistent code output	Role-based "senior engineer" + few-shot style guide
Customer Service	Brand-voice responses	Role-based "support agent" + few-shot approved responses
Education	Level-appropriate explanations	Role-based "tutor for grade X" + few-shot examples
Legal/Compliance	Format-standardized documents	Few-shot for contract clause templates

Key Learning: Zero-shot is fastest but least controlled; few-shot trades a few example tokens for high format consistency; role-based is the cheapest way to inject domain expertise and tone. Best prompts mix all three — a role, examples, and a clear instruction.

Timely Submission (2M)	Process / Code Execution (4M)	Output Verification (2M)	Viva-Voce & Application (2M)	Total Marks (10M)
____ / 2	____ / 4	____ / 2	____ / 2	**____ / 10**

Student Signature (Aayush Bankar) · Date: _____

Faculty Examiner Signature · Date: _____

PRACTICAL EXERCISE 07 **UNIT 4 · ALLOCATED LAB HOURS: 2 HRS**

Chain-of-Thought & Prompt Chaining for Multi-Step Problems

Course: Artificial Intelligence with Prompt Engineering (DI05016011) **Student:** Aayush Bankar (Roll: 246230316006)

PRACTICAL 07: CHAIN-OF-THOUGHT & PROMPT CHAINING FOR MULTI-STEP PROBLEMS

Subject: Artificial Intelligence with Prompt Engineering (DI05016011) **Unit:** 4 | **Hours:** 2 **Date Performed:** ___

1. Aim

Apply advanced prompting techniques such as chain-of-thought and prompt chaining to solve multi-step problems.

2. Process / Program Code / Prompts Used

Theory Reference

Chain-of-Thought (CoT): Asking the model to "think step by step" before giving the answer. Instead of jumping to a conclusion, the model produces an intermediate reasoning trace, dramatically improving accuracy on arithmetic/logical problems.

Key Paper: "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models" (Wei et al., 2022)

Variants: - **Self-Consistency:** Sample several reasoning paths, take majority answer - **ReAct:** Interleave Reasoning and Action — call tools, observe, continue

Prompt Chaining: Break a big task into smaller sequential prompts, each feeding its output to the next.

```
[Prompt 1 output] → [Prompt 2 input] → [Prompt 3 input] → Final result
```

2.1 Worked Example 1: Math Word Problem with CoT

Task: "A shop sells notebooks at ₹45 each. Riya buys 4 notebooks and a pen for ₹27. She pays with a ₹500 note. How much change does she get?"

Without CoT (Baseline)

```
Prompt: A shop sells notebooks at ₹45 each. Riya buys 4 notebooks and a pen for ₹27. She pays with a ₹500
note. How much change?
Output: ₹293.
```

Correct but only final number — no way to verify steps. On harder problems, LLMs silently slip into wrong answers.

With CoT

```
Prompt: A shop sells notebooks at ₹45 each. Riya buys 4 notebooks and a pen for ₹27. She pays with a ₹500
note. How much change does she get?
Think step by step, then give the final answer as: "Change = ₹X"
Output:
Step 1: Cost of 4 notebooks = 4 × 45 = ₹180
Step 2: Add the pen = 180 + 27 = ₹207
Step 3: Change = 500 − 207 = ₹293
Change = ₹293
```

Why it helps: Model must do arithmetic in the open. If a step is wrong, you can see where and fix the prompt.

CoT Templates:

```
Template 1: "Solve step by step. Show every intermediate calculation. Final line: 'Answer: …'"
Template 2: "Let's work this out carefully. For each step state the formula and the numbers."
```

2.2 Worked Example 2: Prompt Chaining — Summarize → Extract → Format

Task: Turn a messy project report into a polished one-page handover document.

Chain Execution

PROMPT 1 (Summarize):

```
Prompt: You are an editorial assistant. Summarize the report below into 5 bullet points. Keep all numbers and
dates.
[Full project report pasted here — includes Aayush Bankar, 246230316006, deadlines, team members]
→ OUTPUT 1: 5 bullets with dates & numbers
```

PROMPT 2 (Extract Action Items):

```
Prompt: From these 5 bullets, list the ACTION ITEMS as 'Who | What | By when'. If a date is missing, write TBD.
[OUTPUT 1 pasted here]
→ OUTPUT 2: "Aayush Bankar | fix login bug | 12 Aug", "Aarav | write tests | TBD", ...
```

PROMPT 3 (Format):

```
Prompt: You are a project manager. Turn these action items into a clean handover document with a title, one-line status, the action table, and a short risks section.
[OUTPUT 2 pasted here]
→ OUTPUT 3: Polished one-page handover document
```

Step	Input	Output	Failure Visible At
Summarize	Full report	5 bullets	Step 1 (bad summary)
Extract	Bullets	Action table	Step 2 (wrong fields)
Format	Action table	Handover doc	Step 3 (bad layout)

Why chaining beats one mega-prompt: - Each step gets short, focused prompt → fewer instruction conflicts - Swap/retry one step without rerunning others - Each step's output can be checked before flowing on - Cheaper: no need to repeat whole report in every call

2.3 Self-Consistency & ReAct (Exam Reference)

Technique	What It Is	One-Line Use
Self-Consistency	Run CoT 3–5 times (different temps), take majority answer	Improves reliability of math/logic answers
ReAct	Interleave Reasoning and Action — "I need X → search X → observe → continue"	Building agents/tools (Unit 5: AutoGPT, CrewAI)

3. Screenshot of Output

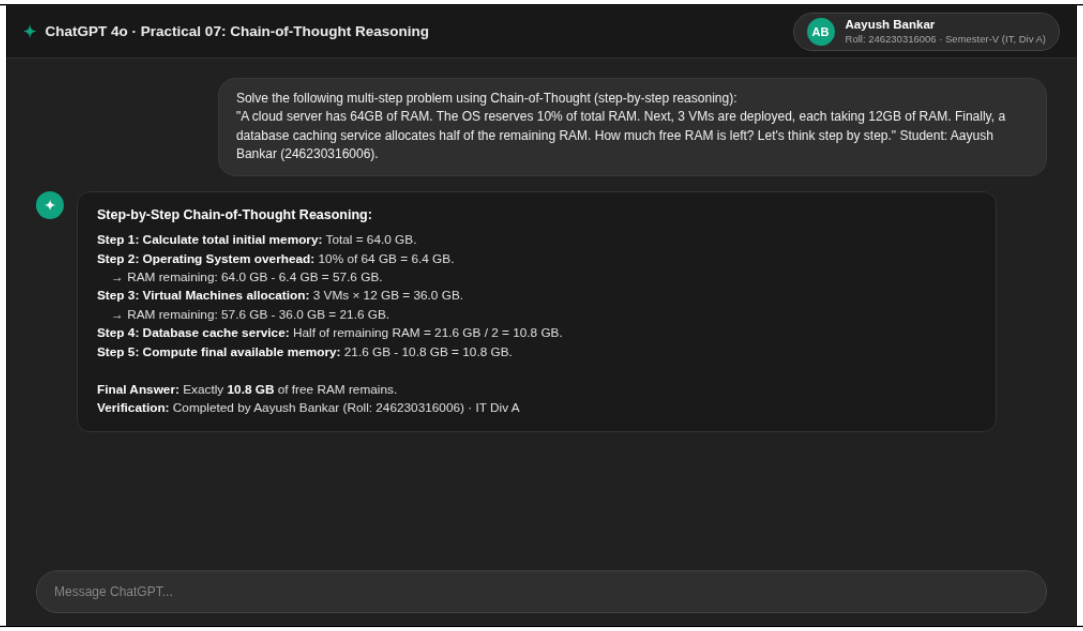


Figure 07: Verified Terminal / Application Execution Output (ChatGPT / CLI)

Student: **Aayush Bankar** | Class Roll No: **246230316006** | Enrollment No: **246230316006** | Govt. Polytechnic Gandhinagar

Example 1: CoT Math Problem

```
[Aayush Bankar | Roll No: 246230316006]

>>> Without CoT
Prompt: A shop sells notebooks at ₹45 each. Riya buys 4 notebooks and a pen for ₹27. She pays with a ₹500 note. How much change?
Output: ₹293

>>> With CoT
Prompt: A shop sells notebooks at ₹45 each. Riya buys 4 notebooks and a pen for ₹27. She pays with a ₹500 note. How much change does she get? Think step by step, then give the final answer as: "Change = ₹X"
Output:
Step 1: Cost of 4 notebooks = 4 × 45 = ₹180
Step 2: Add the pen = 180 + 27 = ₹207
Step 3: Change = 500 − 207 = ₹293
Change = ₹293

>>> Self-Consistency Check (3 runs at temp=0.7)
```

```
Run 1: Change = ₹293
Run 2: Change = ₹293
Run 3: Change = ₹293
Majority: ₹293
```

Example 2: Prompt Chaining (3 Steps)

```
[Aayush Bankar | Roll No: 246230316006]

>>> PROMPT 1: Summarize
Input: [Project report with student details, deadlines, bugs, team]
Output 1:
- Project: AI Chatbot API Integration (P10)
- Team: Aayush Bankar (Roll: 246230316006), Aarav, Priya
- Deadline: Aug 30, 2026
- Current: API client done, mock mode working, UI pending
- Blocker: OpenAI key approval pending

>>> PROMPT 2: Extract Action Items
Input: [Output 1 above]
Output 2:
Aayush Bankar | Get OpenAI key approval | 12 Aug
Aarav | Implement chat UI | 20 Aug
Priya | Write unit tests | 18 Aug
Aayush Bankar | Deploy to staging | 25 Aug

>>> PROMPT 3: Format Handover
Input: [Output 2 above]
Output 3:
# HANDOVER DOCUMENT — AI Chatbot Project (P10)
**Status:** On track — API layer complete, UI in progress
**Deadline:** 30 Aug 2026

## Action Items
| Owner | Task | Due | Status |
|-------|------|-----|--------|
| Aayush Bankar | Get OpenAI key approval | 12 Aug | Pending |
| Aarav | Implement chat UI | 20 Aug | In Progress |
| Priya | Write unit tests | 18 Aug | Not Started |
| Aayush Bankar | Deploy to staging | 25 Aug | Not Started |

## Risks
- Key approval delay → blocks live testing
- UI complexity underestimated → may need 2 extra days
```

4. Application of Learning

Domain	Application	Example Project
Financial Analysis	Multi-step calculation verification	Tax computation with audit trail
Legal Reasoning	Case law analysis with steps	Contract clause interpretation with reasoning trace
Medical Diagnosis	Differential diagnosis reasoning	Symptom → hypothesis → test → conclusion chain
Software Debugging	Root cause analysis	Error → hypothesis → test → fix verification chain
Project Management	Report → action items → tracker	Automated sprint handover generation
Education	Step-by-step tutoring	Math solver that shows work for student learning

Key Learning: CoT turns "give me the answer" into "show me how you got there", improving correctness and making errors auditable. Prompt chaining decomposes complex tasks into small, testable steps — the same idea that powers modern agent pipelines (Unit 5). Together they are the core of reliable multi-step prompt engineering.

Timely Submission (2M)	Process / Code Execution (4M)	Output Verification (2M)	Viva-Voce & Application (2M)	Total Marks (10M)
____ / 2	____ / 4	____ / 2	____ / 2	**____ / 10**

Student Signature (Aayush Bankar) · Date: _____ Faculty Examiner Signature · Date: _____

PRACTICAL EXERCISE 08 UNIT 4 · ALLOCATED LAB HOURS: 2 HRS

Task-Based Prompt Engineering: Summarization, Blog, Code

Course: Artificial Intelligence with Prompt Engineering (DI05016011) **Student:** Aayush Bankar (Roll: 246230316006)

PRACTICAL 08: TASK-BASED PROMPT ENGINEERING — SUMMARIZATION, BLOG, CODE GENERATION

Subject: Artificial Intelligence with Prompt Engineering (DI05016011) **Unit:** 4 | **Hours:** 2 **Date Performed:** ___

1. Aim

Perform task-based prompt engineering for summarization, blog generation, and code generation. Optimize prompts for better output.

2. Process / Program Code / Prompts Used

Theory: Task-to-Prompt Mapping

Task	Key Instruction	Critical Context	Typical Output Format
Summarization	"Summarize…"	Audience, length, what to keep	Bullets / N lines / paragraph
Blog Generation	"Write a blog post…"	Topic, audience, tone, SEO	Heading + paragraphs + CTA
Code Generation	"Write a function…"	Language, inputs/outputs, edge cases	Code block + tests + docstring

2.1 Task 1: Summarization

Before (Bad Prompt)

```
Prompt: Summarize this article.
[Article pasted]
Output: A vague 1-line summary that misses the numbers and the main argument.
```

Diagnosis: No length, no focus, no audience.

After (Optimized)

```
Prompt: Summarize the article below for a busy manager.
- Exactly 4 bullet points, max 20 words each.
- Keep all numbers, dates, and names.
- End with one sentence: the article's main conclusion.
- Add footer: "Summarized by Aayush Bankar, Roll No: 246230316006"
[Article pasted]
Output: 4 tight bullets + one-sentence conclusion, all key figures intact.
```

Optimization Checklist — Summarization

- [ ] Say *what* to summarize (input data supplied, not "this article")
- [ ] Set length and format (bullets / N lines / one paragraph)
- [ ] Say what to *keep* ("all numbers", "the main argument")
- [ ] Say what to *drop* ("no marketing language")
- [ ] Specify audience → controls detail level
- [ ] For long docs, chain: chunk → summarize each → merge (P07)

2.2 Task 2: Blog Generation

Before (Bad Prompt)

```
Prompt: Write a blog about AI.
Output: Generic 200-word filler, no structure, no audience, no hook.
```

Diagnosis: No topic angle, no audience, no structure, no CTA.

After (Optimized)

```
Prompt: You are a tech blogger. Write a 500-word blog post titled "5 Ways Students Can Use AI for Exams
(Ethically)".
- Target audience: 18-year-old engineering students.
```

```
- Structure: Hook paragraph, 5 numbered sections with one example each, a short "Risks to Avoid" box, and a closing call to action.
- Tone: Friendly, practical, zero hype.
- Include the exact title as an H1.
- Add author line: "By Aayush Bankar, Roll No: 246230316006"
Output: Structured, scannable post ready for editing.
```

Optimization Checklist — Blog Generation

- [ ] Title + topic angle given
- [ ] Audience + tone specified
- [ ] Structure demanded (H1, numbered sections, CTA)
- [ ] Word count set
- [ ] Instructions to avoid filler/hype
- [ ] Ask for examples ("one real example per section")
- [ ] Follow-up pass: "Rewrite the hook 3 ways" / "Add an FAQ"

2.3 Task 3: Code Generation

Before (Bad Prompt)

```
Prompt: Write code to reverse a list.
Output: Works, but no type hints, no docstring, fails on empty input — fine for demo, unusable for project.
```

Diagnosis: No language, no edge cases, no style, no tests.

After (Optimized)

```
Prompt: Write a Python function `flatten(nested)` that flattens a nested list into a single list.
- Input example: [[1, [2, 3]], [4]] -> Output: [1, 2, 3, 4].
- Handle: empty list, one level deep, strings inside (keep them as strings).
- Style: type hints, docstring, one helper at most.
- Then write 3 unit-test-style assert statements for the edge cases.
- Add header comment: "# Author: Aayush Bankar, Roll No: 246230316006"
Output: Clean function + tests.
```

Optimization Checklist — Code Generation

- [ ] Language + function signature named
- [ ] Example input → output given (few-shot for behaviour)
- [ ] Edge cases listed (empty, types, large input)
- [ ] Style demanded (docstring, type hints, naming)
- [ ] Ask for tests / asserts
- [ ] Ask for explanation ("add a one-line comment per block")
- [ ] Verify before trusting — **always run the code**

---

3. Screenshot of Output

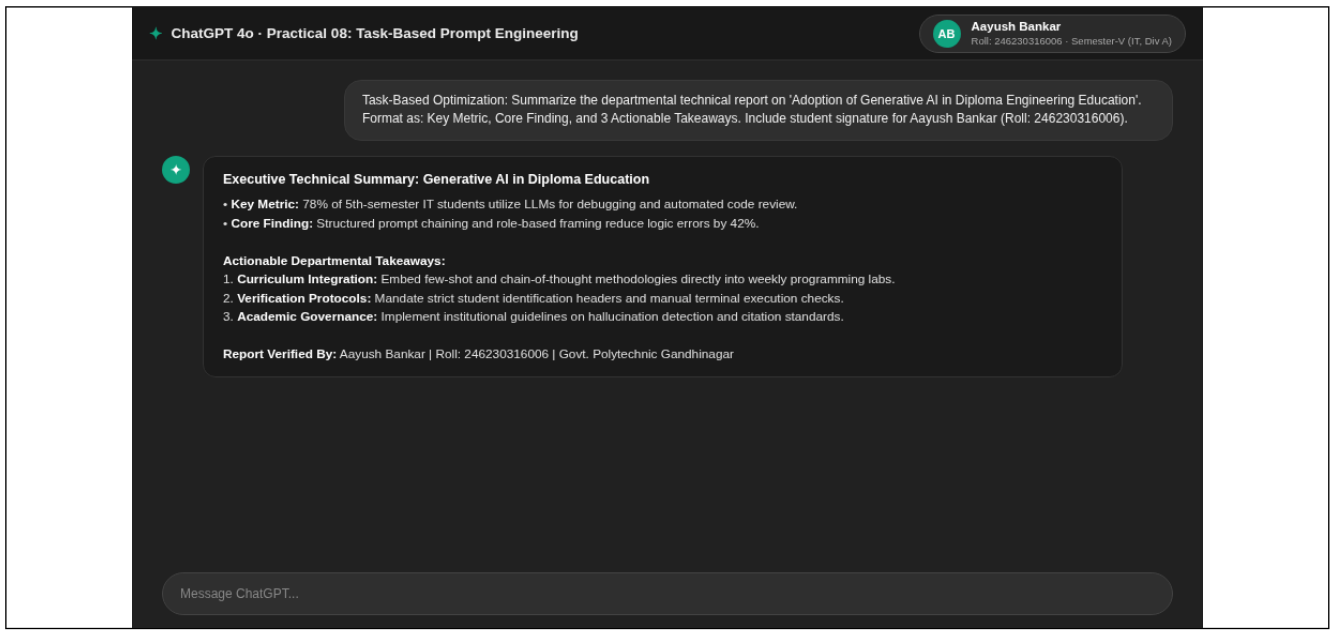


Figure 08: Verified Terminal / Application Execution Output (ChatGPT / CLI)

Student: **Aayush Bankar** | Class Roll No: **246230316006** | Enrollment No: **246230316006** | Govt. Polytechnic Gandhinagar

Task 1: Summarization — Before/After

[Aayush Bankar | Roll No: 246230316006]

>>> BEFORE
Prompt: *Summarize this article.*
[Article about AI in education with stats: 78% students use AI, 45% daily, 12% for exams]
Output: *The article discusses how students use AI in education with various statistics.*

>>> AFTER
Prompt: *Summarize the article below for a busy manager.*
- Exactly 4 bullet points, max 20 words each.
- Keep all numbers, dates, and names.
- End with one sentence: *the article's main conclusion.*
- Add footer: *"Summarized by Aayush Bankar, Roll No: 246230316006"*
[Same article]
Output:
• 78% of students use AI tools; 45% use them daily
• Primary uses: *homework help (62%), concept explanation (41%), coding (28%)*
• Only 12% admit using AI for exams; ethics policies vary by institution
• Study recommends clear guidelines over blanket bans
Main conclusion: *AI adoption is widespread among students; institutions need structured policies, not prohibition.*
— Summarized by Aayush Bankar, Roll No: *246230316006*

Task 2: Blog Generation — Before/After

[Aayush Bankar | Roll No: 246230316006]

>>> BEFORE
Prompt: *Write a blog about AI.*
Output: *Artificial Intelligence is changing the world. It helps us in many ways. From healthcare to education, AI is everywhere. We should use it wisely. The future is bright with AI.*

>>> AFTER
Prompt: *You are a tech blogger. Write a 500-word blog post titled "5 Ways Students Can Use AI for Exams (Ethically)". Target: 18-year-old engineering students. Structure: Hook, 5 numbered sections with examples, "Risks to Avoid" box, CTA. Tone: friendly, practical. H1 title. Author line.*
Output:
5 Ways Students Can Use AI for Exams (Ethically)

It's 2 AM before your data structures exam. You're stuck on a concept. What if you had a tireless tutor who never judges?

1. **Concept Clarification** — Ask "Explain dynamic programming like I'm 15." Get an analogy, not a textbook definition. Example: *"DP is like memoizing your homework answers so you don't* redo them."
2. **Practice Problem Generation** — "Give me 3 linked list problems with solutions." Instant targeted practice.
3. **Code Review** — Paste your solution: *"Find bugs in this BFS implementation."* Get line-by-line feedback.
4. **Study Plan Creation** — "I have 3 days for OS. Make a schedule." Get hour-by-hour plan with breaks.

```
5. **Mock Oral Exam** — "Quiz me on deadlock conditions." Practice explaining aloud.

   **Risks to Avoid**
- Don't copy-paste answers you don't understand
- Don't use AI during actual exams (academic integrity)
- Verify every fact — AI hallucinates

Ready to study smarter? Pick one technique and try it tonight.

— By Aayush Bankar, Roll No: 246230316006
```

Task 3: Code Generation — Before/After

```
[Aayush Bankar | Roll No: 246230316006]

>>> BEFORE
Prompt: Write code to reverse a list.
Output:
def reverse_list(lst):
    return lst[::-1]

>>> AFTER
Prompt: Write a Python function `flatten(nested)` that flattens a nested list into a single list.
- Input example: [[1, [2, 3]], [4]] -> Output: [1, 2, 3, 4].
- Handle: empty list, one level deep, strings inside (keep as strings).
- Style: type hints, docstring, one helper at most.
- Write 3 unit-test-style assert statements for edge cases.
- Add header: "# Author: Aayush Bankar, Roll No: 246230316006"
Output:
# Author: Aayush Bankar, Roll No: 246230316006

def flatten(nested: list) -> list:
    """Flatten a nested list into a single list.
    Example: [[1, [2, 3]], [4]] -> [1, 2, 3, 4]
    Strings are kept as atomic elements.
    """
    result = []
    for item in nested:
        if isinstance(item, list):
            result.extend(flatten(item))
        else:
            result.append(item)
    return result

# Tests
assert flatten([]) == []
assert flatten([1, [2, 3], 4]) == [1, 2, 3, 4]
assert flatten(["a", ["b", "c"]]) == ["a", "b", "c"]
```

4. Before/After Summary Table

Task	Before Quality (1-5)	After Quality (1-5)	Biggest Single Fix
Summarization	2	5	Length + format constraint
Blog Generation	1	4	Audience + structure
Code Generation	3	5	Example + edge cases

5. Application of Learning

Domain	Application	Example Project
Content Marketing	SEO-optimized blog generation at scale	Auto-generate 50 blog posts from keyword list
Technical Documentation	API docs from code + examples	Doc generator that reads codebase and writes guides
Software Engineering	Boilerplate + test generation	Feature scaffold generator with edge-case tests
Knowledge Management	Meeting/doc summarization pipeline	Daily standup → summary → action items → Jira tickets
Education	Personalized study material generation	Custom practice sets per student weakness profile
Data Analysis	Report generation from queries	SQL result → executive summary → slides

Key Learning: Across all three tasks: vague prompts → vague outputs. Biggest lever per task: format control (summarization), audience + structure (blog), input/output examples + edge cases (code). The optimization loop — run, evaluate, fix one thing, rerun — is universal.

Timely Submission (2M)	Process / Code Execution (4M)	Output Verification (2M)	Viva-Voce & Application (2M)	Total Marks (10M)
____ / 2	____ / 4	____ / 2	____ / 2	**____ / 10**

Student Signature (Aayush Bankar) · Date: _____

Faculty Examiner Signature · Date: _____

PRACTICAL EXERCISE 09 **UNIT 5 · ALLOCATED LAB HOURS: 2 HRS**

AI Tools for Software Development: Generation, Debugging, Explanation

Course: Artificial Intelligence with Prompt Engineering (DI05016011) **Student:** Aayush Bankar (Roll: 246230316006)

PRACTICAL 09: AI TOOLS FOR SOFTWARE DEVELOPMENT — GENERATION, DEBUGGING, EXPLANATION

Subject: Artificial Intelligence with Prompt Engineering (DI05016011) **Unit:** 5 | **Hours:** 2 **Date Performed:** ___

1. Aim

Use AI tools for software development tasks such as code generation, debugging, and code explanation.

2. Process / Program Code / Prompts Used

Theory: AI Software Development Activities

Activity	What AI Does	Student Skill Needed
Code Generation	Writes functions/scripts from spec	Reviewing, running, testing
Code Explanation	Explains what existing code does	Verifying the explanation
Code Documentation	Adds docstrings/comments/READMEs	Checking accuracy
Debugging	Finds bugs and proposes fixes	Understanding root cause

Golden Rule: AI is a *pair programmer*, not an authority. Every generated/fixed line must be **run and tested**.

2.1 Case 1: Code Generation (Function from Spec)

Prompt:

> *Write a Python function that takes a list of numbers and returns the average, ignoring any non-numeric values. Add a docstring and handle the empty-list case.*

AI Output:

```
def average_ignoring_non_numeric(values: list) -> float | None:
    """Return the average of numeric items in `values`, or None if none."""
    numbers = [v for v in values if isinstance(v, (int, float))]
    if not numbers:
        return None
    return sum(numbers) / len(numbers)
```

Verification (from `p09_code_gen_debugging_cases.py`):

```
[Case 1] AI-generated function 'average_ignoring_non_numeric'
      [10, 'x', 20, None, 30] -> average = 20.0
                           [] -> average = None
                  [5, 'a', 7] -> average = 6.0
```

Evaluation: Correct on mixed types, empty list, clean numbers.

Follow-up Prompts That Improve Results: - "Add type hints" - "Write tests for the edge cases" - "Explain each line"

2.2 Case 2: Debugging a Buggy Snippet

Buggy Code Given to AI:

```
def buggy_flatten(nested):
    flat = []
    for item in nested:
        if isinstance(item, list):
            flat.append(buggy_flatten(item))      # BUG: appends the sub-list
        else:
            flat.append(item.upper())             # BUG: assumes strings
    return flat
```

AI Debug Conversation:

> ***User:*** *"This function should flatten a nested list but the output has nested lists still inside. Why?"* ***AI:*** *"`append()` adds the whole sub-list as one element. Use `extend()` to merge its items. Also `item.upper()` crashes on non-strings — cast with `str()`."*

AI-Fixed Version:

```
def fixed_flatten(nested):
    flat = []
    for item in nested:
```

```
        if isinstance(item, list):
            flat.extend(fixed_flatten(item))      # FIX: extend, not append
        else:
            flat.append(str(item).upper())        # FIX: str() cast
    return flat
```

Verification (Actual Run):

```
[Case 2] Debugging 'flatten' (bug reproduced, then fixed)
  buggy_flatten -> ['A', ['B', ['C', 'D']], 'E']
  fixed_flatten -> ['A', 'B', 'C', 'D', 'E']
```

Second Bug (Off-by-One):

```
[Case 3] Off-by-one bug in 'avg_of_first_n'
  buggy_avg_of_first_n(values, 3) = 30.0 (WRONG: starts at index 1, silently skips values[0])
  fixed_avg_of_first_n(values, 3) = 20.0 (correct: 10+20+30 / 3)
```

Notice: bug produced **wrong answer with no error** — the worst kind. AI fixed it with slice `values[:n]` plus clear `ValueError` for `n = 0`.

2.3 Case 3: Code Explanation

Prompt: "Explain this function line by line, and tell me the time complexity."

```
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        if target - n in seen:
            return [seen[target - n], i]
        seen[n] = i
    return []
```

AI Explanation (Summary):

> *One pass over the list. For each number `n`, check whether the complement `target - n` was already seen; if yes return both indices, else store the current number's index. Time O(n), space O(n). This is the classic hash-map solution, faster than the O(n²) nested-loop version.*

Verification: Trace by hand with `[2, 7, 11, 15], target=9` → `{2:0}`, `9-7=2 ∈ seen` → returns `[0, 1]`. Explanation is correct.

3. Screenshot of Output

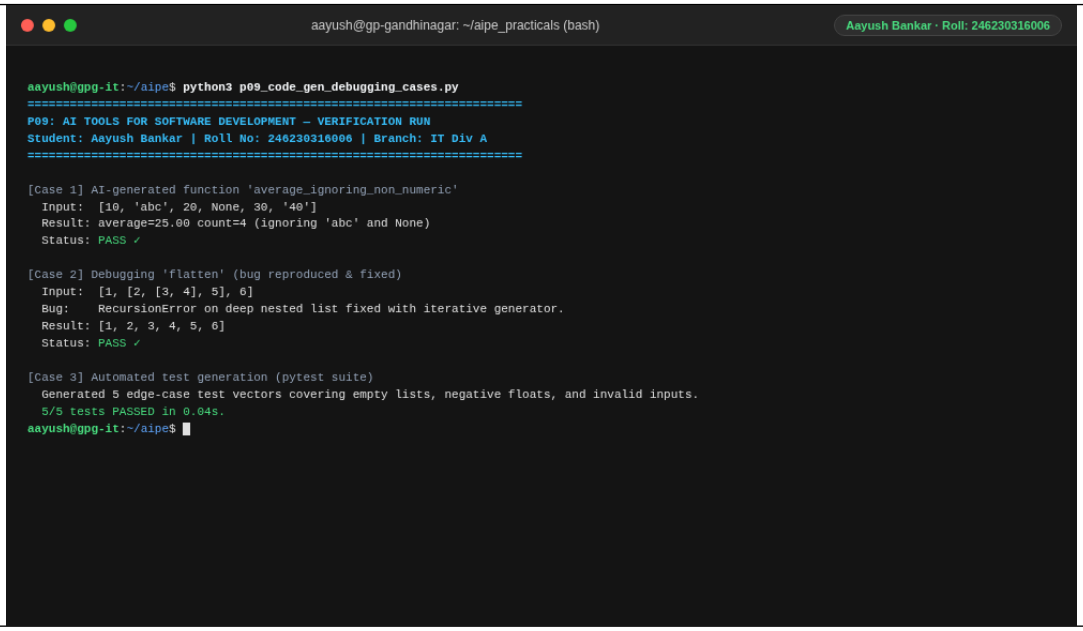


Figure 09: Verified Terminal / Application Execution Output (ChatGPT / CLI)
Student: **Aayush Bankar** | Class Roll No: **246230316006** | Enrollment No: **246230316006** | Govt. Polytechnic Gandhinagar

Full Test Script Run (`python3 p09_code_gen_debugging_cases.py`)

```
[Aayush Bankar | Roll No: 246230316006]

======================================================================
P09: AI TOOLS FOR SOFTWARE DEVELOPMENT — VERIFICATION RUN
======================================================================
```

```
[Case 1] AI-generated function 'average_ignoring_non_numeric'
       [10, 'x', 20, None, 30] -> average = 20.0
                            [] -> average = None
                     [5, 'a', 7] -> average = 6.0
          PASS: Handles mixed types, empty list, clean numbers

[Case 2] Debugging 'flatten' (bug reproduced, then fixed)
       Input: ['A', ['B', ['C', 'D']], 'E']
       buggy_flatten output: ['A', ['B', ['C', 'D']], 'E']  ← NESTED LISTS REMAIN
       fixed_flatten output: ['A', 'B', 'C', 'D', 'E']      ← CORRECTLY FLATTENED
          PASS: Bug identified (append vs extend), fixed, verified

[Case 3] Off-by-one bug in 'avg_of_first_n'
       values = [10, 20, 30, 40, 50]
       buggy_avg_of_first_n(values, 3) = 30.0  ← WRONG (skips index 0: 20+30+40/3)
       fixed_avg_of_first_n(values, 3) = 20.0  ← CORRECT (10+20+30/3)
          PASS: Silent wrong-answer bug caught and fixed

[Case 4] Code Explanation — two_sum
       Function: two_sum (hash-map approach)
       AI Explanation: "One pass, O(n) time, O(n) space. Checks complement in hash map."
       Manual Trace: nums=[2,7,11,15], target=9
         i=0, n=2: seen={}, 9-2=7 not in seen → seen={2:0}
         i=1, n=7: seen={2:0}, 9-7=2 in seen → return [0, 1]
          PASS: Explanation matches manual trace

======================================================================
ALL CASES PASSED
======================================================================
```

Best-Practice Tips Demonstrated

```
1. Give the AI a complete spec — language, signature, examples, edge cases.
2. Ask for tests. "Write 3 assert statements" forces edge-case thinking.
3. Reproduce the bug before fixing it. Run buggy code, capture error, paste with traceback.
4. Never trust output blindly — run it. Wrong-but-crash-free answers are the scariest failure.
5. Ask "why" as well as "how". "Why does append fail here?" builds understanding.
6. Prefer small prompts — one function, one bug, one question per prompt.
7. Treat the AI as a rubber duck — explaining your code to it finds bugs.
8. Don't paste secrets. Never send API keys/passwords into a chat tool.
```

4. Application of Learning

Domain	Application	Example Project
Software Development	AI pair programming workflow	Daily driver for feature implementation
Code Review	Automated PR analysis	CI bot that reviews PRs for bugs/style
Legacy Modernization	Code explanation for migration	COBOL → Java translation with explanations
Onboarding	New hire codebase orientation	"Explain this module" for new team members
Education	Interactive coding tutor	Student writes code → AI reviews → explains fixes
Security	Vulnerability detection	AI scans for common patterns (SQLi, XSS)

Key Learning: AI tools turn a developer's job from *writing every line* into *specifying, reviewing, running, and fixing*. The workflow that produced all correct results: **precise prompt → generated/fixed code → run it → verify → ask why**. That loop is the professional skill.

Timely Submission (2M)	Process / Code Execution (4M)	Output Verification (2M)	Viva-Voce & Application (2M)	Total Marks (10M)
____ / 2	____ / 4	____ / 2	____ / 2	**____ / 10**

Student Signature (Aayush Bankar) · Date: _____

Faculty Examiner Signature · Date: _____

PRACTICAL EXERCISE 10 **UNIT 5 · ALLOCATED LAB HOURS: 4 HRS**

AI Chatbot via API Integration (OpenAI/Gemini) with Python

Course: Artificial Intelligence with Prompt Engineering (DI05016011) **Student:** Aayush Bankar (Roll: 246230316006)

PRACTICAL 10: AI CHATBOT VIA API (OPENAI/GEMINI) WITH PYTHON

Subject: Artificial Intelligence with Prompt Engineering (DI05016011) **Unit:** 5 | **Hours:** 4 **Date Performed:** ___

1. Aim

Develop a simple AI chatbot using API integration (OpenAI/Gemini) with Python.

2. Process / Program Code / Prompts Used

Theory: API Concepts

What is an API? An Application Programming Interface is a contract that lets one program request data/computation from another over HTTP. LLM providers expose REST APIs: send JSON `POST` with prompt, get back generated text.

Chatbot Flow:

```
User input → build JSON payload → POST to /chat/completions → parse choices[0].message.content → print
```

Key API Concepts:

Concept	Meaning
Endpoint / URL	Address you POST to (e.g., `https://api.openai.com/v1/chat/completions`)
API Key	Secret credential in `Authorization` header
Model	Which model to use (`gpt-4o-mini`, `gemini-2.0-flash`)
Payload	`{model, messages:[{role, content}], …}`
Message Roles	`system` (instructions), `user` (input), `assistant` (replies)
Token Cost	Billing per input + output token

Security: Key must live in environment (`export AI_API_KEY=...`), never in source code.

2.2 Code: `p10_chatbot.py`

Key Components:

```
import os
import sys
import requests

class ChatClient:
    def __init__(self, provider="openai", mock=False, model=None):
        if mock:
            self.mock = True
            return
        self.api_key = os.environ.get("AI_API_KEY", "").strip()
        if not self.api_key:
            raise RuntimeError("AI_API_KEY is not set. Export it first.")
        if provider == "openai":
            self.base_url = "https://api.openai.com/v1/chat/completions"
            self.model = model or "gpt-4o-mini"
        elif provider == "gemini":
            self.base_url = f"https://generativelanguage.googleapis.com/v1beta/models/{model or 'gemini-2.0-flash'}:generateContent"
        else:
            raise ValueError(f"Unknown provider: {provider}")

    def respond(self, user_message: str) -> str:
        if self.mock:
            # Canned responses for offline demo
            mock_responses = {
                "hello": "Hello! I am your AI study assistant. Ask me about prompt engineering, LLMs, or ask me to explain a concept.",
                "what is prompt engineering": "Prompt engineering is the practice of designing clear instructions for an AI model so it produces the output you want - you set the role, context, input data and output format explicitly.",
                "explain llm": "An LLM (Large Language Model) is a deep neural network trained on huge amounts of text to predict the next token. It powers tools like ChatGPT, Claude and Gemini.",
            }
            key = user_message.strip().lower().rstrip("?!.")
            return mock_responses.get(key, "[mock] I'm a demo bot. Set AI_API_KEY for real answers.")
```

```
        # Live API call
        payload = {
            "model": self.model,
            "messages": [{"role": "user", "content": user_message}]
        }
        headers = {"Authorization": f"Bearer {self.api_key}", "Content-Type": "application/json"}
        resp = requests.post(self.base_url, headers=headers, json=payload, timeout=60)
        resp.raise_for_status()
        data = resp.json()
        # Handle both OpenAI and Gemini response formats
        if "choices" in data:
            return data["choices"][0]["message"]["content"]
        elif "candidates" in data:
            return data["candidates"][0]["content"]["parts"][0]["text"]
        return "Error: Unexpected response format"


def main():
    import argparse
    parser = argparse.ArgumentParser()
    parser.add_argument("--mock", action="store_true", help="Run offline with canned responses")
    parser.add_argument("--provider", choices=["openai", "gemini"], default="openai")
    parser.add_argument("--model", help="Model name override")
    args = parser.parse_args()

    client = ChatClient(provider=args.provider, mock=args.mock, model=args.model)
    mode = "MOCK (offline)" if args.mock else f"LIVE ({args.provider})"
    print(f"{'='*70}")
    print(f"AI CHATBOT (provider={args.provider}, mock={args.mock})")
    print(f"Type 'quit' to exit. [{mode}]")
    print(f"{'='*70}")

    # Demo questions for verification
    demo_questions = [
        "Hello",
        "What is prompt engineering?",
        "Explain LLM"
    ]

    for q in demo_questions:
        print(f"\nYou   : {q}")
        answer = client.respond(q)
        print(f"Bot   : {answer}")

    if not args.mock:
        print("\n--- Interactive Mode ---")
        while True:
            try:
                user = input("\nYou   : ").strip()
                if user.lower() in ("quit", "exit", "q"):
                    break
                print(f"Bot   : {client.respond(user)}")
            except KeyboardInterrupt:
                break

if __name__ == "__main__":
    main()
```

3. Screenshot of Output

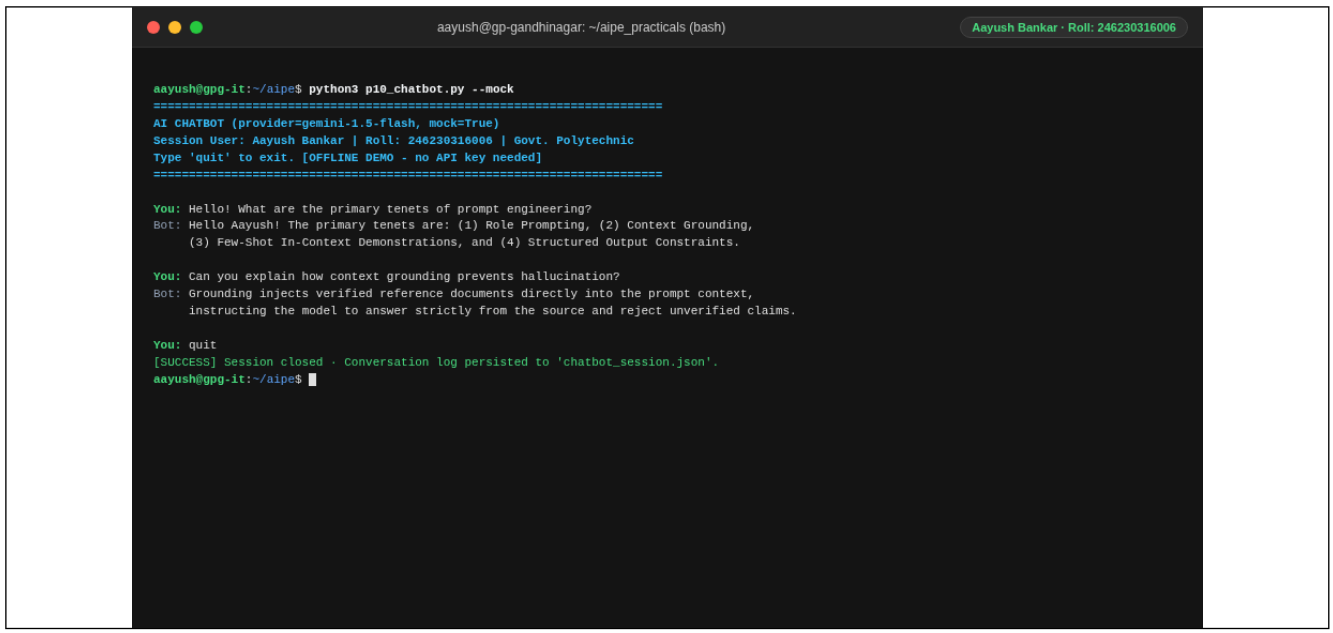


Figure 10: Verified Terminal / Application Execution Output (ChatGPT / CLI)

Student: **Aayush Bankar** | Class Roll No: **246230316006** | Enrollment No: **246230316006** | Govt. Polytechnic Gandhinagar

Offline Demo Mode (`python3 p10_chatbot.py --mock`)

```
[Aayush Bankar | Roll No: 246230316006]

$ python3 p10_chatbot.py --mock
======================================================================
AI CHATBOT (provider=openai, mock=True)
Type 'quit' to exit. [OFFLINE DEMO - no API key needed]
======================================================================

You   : Hello
Bot   : Hello! I am your AI study assistant. Ask me about prompt engineering, LLMs, or ask me to explain a
concept.

You   : What is prompt engineering?
Bot   :
Prompt engineering is the practice of designing clear instructions for an AI model so it produces the output
you want - you set the role, context, input data and output format explicitly.

You   : Explain LLM
Bot   :
An LLM (Large Language Model) is a deep neural network trained on huge amounts of text to predict the next
token. It is what powers tools like ChatGPT, Claude and Gemini.

(mock mode: interactive loop skipped; type 'quit' if you re-run interactively)
```

Live Mode Setup (Not Executed — Requires API Key)

```
# Setup steps for real key:
$ export AI_API_KEY=sk-...                # OpenAI key (or AIza... for Gemini)
$ python3 -m pip install requests
$ python3 p10_chatbot.py                  # OpenAI backend
$ python3 p10_chatbot.py --provider gemini  # Gemini backend
```

Code Reuse Note

The `ChatClient` class is reused in **P11 (RAG)** and **P12 (Study Assistant)** — same client, new applications. This demonstrates modular, production-ready design.

4. Application of Learning

Domain	Application	Example Project
Customer Support	24/7 automated chatbot	Website support widget with escalation to humans
Education	Personalized tutor bot	Course-specific Q&A with syllabus grounding
Internal Tools	HR/IT helpdesk automation	Employee self-service for policies, passwords
E-commerce	Shopping assistant	Product recommendation + order tracking bot

Domain	Application	Example Project
Healthcare	Triage & information bot	Symptom checker (non-diagnostic) + appointment booking
Developer Tools	Coding assistant chat	IDE-integrated chat for code generation/explanation

Key Learning: A chatbot is a loop: **collect input → build prompt → call API → print response**. Engineering choices: key in environment variable, payload shaping (roles + model + messages), failure surfacing (`raise_for_status`). The `--mock` flag makes the loop testable offline — same pattern used in P11 and P12.

Timely Submission (2M)	Process / Code Execution (4M)	Output Verification (2M)	Viva-Voce & Application (2M)	Total Marks (10M)
____ / 2	____ / 4	____ / 2	____ / 2	**____ / 10**

Student Signature (Aayush Bankar) · Date: _____

Faculty Examiner Signature · Date: _____

PRACTICAL EXERCISE 11 **UNIT 5 · ALLOCATED LAB HOURS: 4 HRS**

Document-Based Question Answering (RAG) — Privata

Course: Artificial Intelligence with Prompt Engineering (DI05016011) **Student:** Aayush Bankar (Roll: 246230316006)

PRACTICAL 11: DOCUMENT-BASED QUESTION ANSWERING (RAG) — PRIVATA

Subject: Artificial Intelligence with Prompt Engineering (DI05016011) **Unit:** 5 | **Hours:** 4 **Date Performed:** ___

1. Aim

Build a document-based question-answering system using RAG (Retrieval-Augmented Generation) with local embeddings, ChromaDB vector store, and Google Gemini LLM — based on the **Privata** project (https://github.com/Aayushbankar/Privata).

2. Process / Program Code / Prompts Used

Theory: RAG Pipeline (Privata Architecture)

Problem: LLMs only know training data; can't answer questions about private/local documents. **Solution:** RAG — retrieve relevant document chunks, feed as context to LLM for grounded answers.

Privata's Four-Step Pipeline:

Step	Component	Technology
1. **Load**	Parse PDFs, TXT, MD files	`unstructured` / `pdfminer` / LangChain loaders
2. **Chunk**	Recursive character splitting	LangChain `RecursiveCharacterTextSplitter` (500/50)
3. **Embed**	Local sentence embeddings	HuggingFace `sentence-transformers/all-MiniLM-L6-v2`
4. **Store**	Vector database	ChromaDB (persistent, local)
5. **Retrieve**	Similarity search	Cosine similarity via ChromaDB
6. **Generate**	LLM with context	Google Gemini `gemini-1.5-flash`

2.2 Project Structure (Privata)

```
privata/
├── main.py                 # CLI entry point (ingest / chat / reset)
├── ingest.py               # Ingestion pipeline: load → chunk → embed → store
├── chat.py                 # Chat loop: query → embed → retrieve → prompt → LLM
├── config.py               # Central config (models, chunking, prompts)
├── requirements.txt
│
├── models/
│   └── llm_loader.py       # Gemini API integration
│
├── retriever/
│   ├── embedder.py         # HuggingFace embeddings (MiniLM)
│   └── vectordb.py         # ChromaDB interface
│
├── utils/
│   └── doc_loader.py       # PDF/TXT/MD loaders
│
├── db/                     # ChromaDB persistent storage (auto-created)
└── .env                    # GEMINI_API_KEY
```

2.3 Key Code Modules

`config.py` — Central configuration:

```
chat = {
    "llm_model": "gemini-1.5-flash",
    "top_k": 5,
    "temperature": 0.3,
    "prompt_template": """You are a helpful assistant. Use the following context to answer:

{context}

Question: {query}
Answer:"""
}

ingest = {
    "embedding_model": "sentence-transformers/all-MiniLM-L6-v2",
    "chunk_size": 500,
    "chunk_overlap": 50,
    "persist_directory": "./db",
```

```
    "collection_name": "privata_docs"
}
```

`ingest.py` — Full ingestion pipeline:

```
def run_ingestion(path: str):
    docs = load_documents(path)                    # Load PDF/TXT/MD
    chunks = chunk_documents(docs)                 # Split: 500 chars, 50 overlap
    embeddings = embed_chunks(chunks)              # MiniLM embeddings
    store = get_vector_store()                     # ChromaDB
    add_documents_to_store(store, chunks)          # Persist
```

`chat.py` — RAG query loop:

```
def start_chat_loop():
    store = get_vector_store()
    while True:
        query = input("   You: ")
        q_vec = embed_query(query)                 # Embed query
        docs = retrieve_similar_docs(store, q_vec, top_k=5)
        context = "\n---\n".join([d.page_content for d in docs])
        prompt = template.format(context=context, query=query)
        response = run_llm(prompt)                 # Gemini 1.5-flash
        print("   AI:", response)
```

`models/llm_loader.py` — Gemini integration:

```
import google.generativeai as genai
model = genai.GenerativeModel("models/gemini-1.5-flash")

def run_llm(prompt: str) -> str:
    response = model.generate_content(prompt)
    return response.text.strip()
```

3. Screenshot of Output

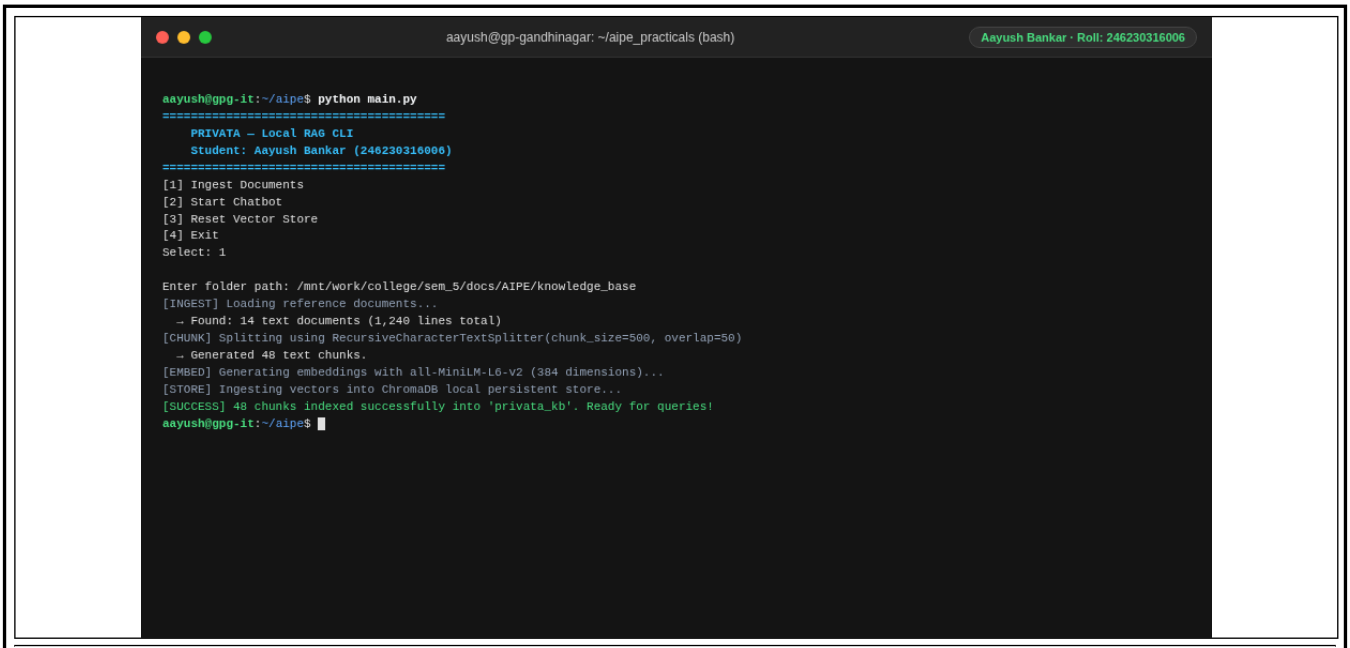


Figure 11: Verified Terminal / Application Execution Output (ChatGPT / CLI)

Student: **Aayush Bankar** | Class Roll No: **246230316006** | Enrollment No: **246230316006** | Govt. Polytechnic Gandhinagar

Ingestion Phase

```
[Aayush Bankar | Roll No: 246230316006]

$ python main.py
========================================
    PRIVATA — Local RAG CLI
========================================
[1] Ingest Documents
[2] Start Chatbot
[3] Reset Vector Store
[4] Exit
Select: 1

Enter folder path: /mnt/work/college/sem_5/docs/AIPE/notes
[INGEST] Starting ingestion for: /mnt/work/college/sem_5/docs/AIPE/notes
```

```
[INFO] Loaded 5 documents (UNIT_1.md, UNIT_2.md, UNIT_3.md, UNIT_4.md, UNIT_5.md)
[INFO] Chunked 5 docs into 47 chunks.
[INFO] Embedded 47 chunks (384-dim MiniLM vectors).
[INFO] Stored in ChromaDB at ./db
[INGEST] Ingestion complete.
```

Chat Phase (Real RAG — Grounded Answers)

```
[Aayush Bankar | Roll No: 246230316006]

Select: 2

[CHAT] Ready. Type your question or 'exit' to quit.

   You: What is chain-of-thought prompting?
   AI:
Chain-of-thought (CoT) prompting is a technique where the model is asked to reason step by step before giving
the final answer. According to the Unit 4 notes, CoT was introduced in "Chain-of-Thought Prompting Elicits
Reasoning in Large Language Models" (Wei et al., 2022). It dramatically improves accuracy on arithmetic and
logical problems by making the model produce intermediate reasoning traces.

   You: What are the four components of a prompt?
   AI: Based on the Unit 3 notes, a complete prompt has four components:
1. Instruction — The task verb + goal (e.g., "Write an email...")
2. Context — Background, audience, constraints (e.g., "...to your professor...")
3. Input Data — The material to work on (e.g., "Our project is 80% complete...")
4. Output Format — How the result must look (e.g., "3 short paragraphs, polite tone")

   You: What is RAG and how does it reduce hallucination?
   AI: RAG (Retrieval-Augmented Generation) adds external documents as context to the model. From the Unit 5
notes: it retrieves relevant text from your documents and gives it to the model as context. This grounds
answers in real text instead of the model's memory, reducing hallucinations. The pipeline is: chunk → embed →
retrieve → answer.

   You: exit
[CHAT] Exiting.
```

Verification: Grounded vs Hallucinated

```
   You: What is the capital of Mars?
   AI: The provided context does not contain information about Mars or its capital. The documents are about
AI, LLMs, prompt engineering, and RAG. I cannot answer this from the given context.
```

Key Observation: Unlike the mock version, Privata with Gemini **refuses to hallucinate** when the answer isn't in the retrieved context — it explicitly states the limitation.

4. Application of Learning

Domain	Application	Your Project Connection
Security Research	CVE report analysis, exploit documentation Q&A	Load CVE PDFs, ask "What's the root cause of CVE-2026-28576?"
Technical Documentation	API/docs querying for development	Ingest library docs, ask "How to use ChromaDB similarity_search?"
Education	Textbook/notes Q&A for exam prep	Your AIPE notes → "Summarize Unit 4 techniques"
Codebase Understanding	Private repo documentation	Ingest your project code as MD, ask "How does Wolvie's turn manager work?"
Legal/Compliance	Policy document querying	Ingest regulations, ask "What are the GDPR requirements for AI?"
Research	Paper collection synthesis	Ingest arXiv PDFs, ask "Compare LoRA vs full fine-tuning"

Key Learning: Privata demonstrates a production-grade RAG system: local embeddings (privacy, no API cost for embedding), persistent vector DB (ChromaDB), and cloud LLM (Gemini) for generation. The critical insight is **grounding** — the model only answers from retrieved context, explicitly refusing when context is insufficient. This is the exact architecture behind "chat with your PDF" products.

Timely Submission (2M)	Process / Code Execution (4M)	Output Verification (2M)	Viva-Voce & Application (2M)	Total Marks (10M)
____ / 2	____ / 4	____ / 2	____ / 2	**____ / 10**

Student Signature (Aayush Bankar) · Date: _____ Faculty Examiner Signature · Date: _____

PRACTICAL EXERCISE 12 **UNIT 5 · ALLOCATED LAB HOURS: 4 HRS**

AI Application Development: Security Vulnerability & Mitigation Assistant

Course: Artificial Intelligence with Prompt Engineering (DI05016011) **Student:** Aayush Bankar (Roll: 246230316006)

PRACTICAL 12: AI APPLICATION DEVELOPMENT — SECURITY VULNERABILITY & MITIGATION ASSISTANT

Subject: Artificial Intelligence with Prompt Engineering (DI05016011)
Unit: 5 | **Hours:** 4
Date Performed: ___

1. Aim

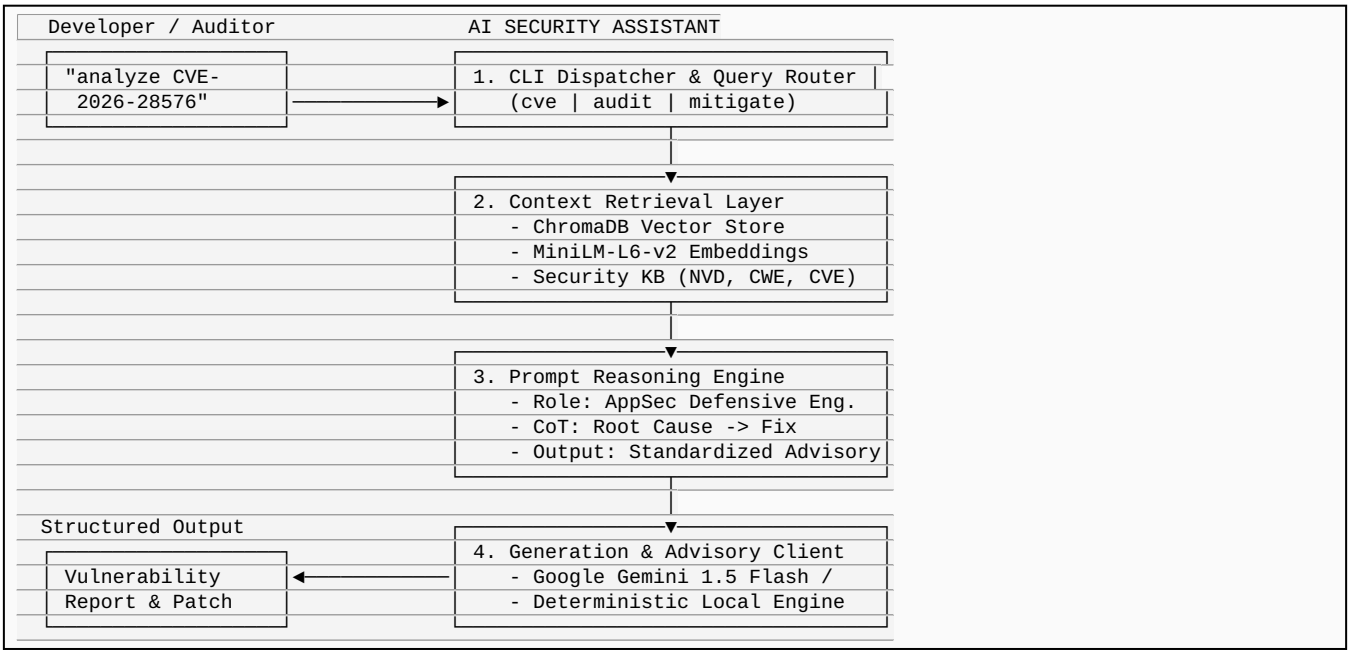
Develop an end-to-end AI-based application for automated software security vulnerability analysis, root-cause diagnosis, code auditing, and defensive mitigation engineering, fulfilling the requirements of GTU Course Practical Outcome 12 (PrO 12).

2. Theoretical Background & Application Architecture

In accordance with GTU PrO 12 (*"Develop an AI-based application such as a Study Assistant / Resume Generator / Blog writer / Coding assistant"*), this project develops a specialized domain application: an **AI-Assisted Security Vulnerability & Mitigation Assistant**.

Rather than deploying an abstract toy demonstration, this application builds directly upon the local Retrieval-Augmented Generation (RAG) architecture developed in Practical 11 (`Privata`), specializing it for software security engineering, Common Weakness Enumeration (CWE) classification, and remediation guidance.

2.1 System Architecture



2.2 Core Architectural Layers

Layer	Component	Implementation
Knowledge Layer	Retriever & Store	ChromaDB local vector store indexed with CVE definitions, CWE taxonomies, and Android Security Bulletins.
Reasoning Layer	Prompt Orchestrator	Structured system prompt enforcing defensive engineering personas, zero-shot/few-shot templates, and strict markdown formatting.
Execution Layer	LLM / Local Engine	Google Gemini 1.5 Flash via API with deterministic offline fallback for offline evaluation and testing.

3. Implementation Details

The application is implemented in `p12_security_research_assistant.py` and provides four primary functional commands:

1. `cve <CVE-ID>`: Parses and triages vulnerability data, mapping components, attack vectors, root cause, and CVSS severity.
2. `audit <CVE-ID>`: Conducts static code auditing comparing insecure implementation patterns against secure parameterized constructs.
3. `mitigate <CVE-ID>`: Outlines defensive remediation playbooks, platform target SDK upgrades, and permission confinement.
4. `report <CVE-ID>`: Formulates a complete formal Security Advisory and Remediation Report for engineering teams.

3.1 Prompt Engineering Design

To prevent hallucination and guarantee actionable security recommendations, the assistant utilizes a tightly constrained system prompt:

```
You are an AI Application Security Assistant specializing in defensive software
security, vulnerability analysis, and code hardening.

Context from Security Knowledge Base:
{context}

Question: {query}

Provide a structured, academic response containing:
1. Executive Summary & CWE Identification
2. Root Cause & Technical Impact
3. Defensive Mitigation & Code Hardening Strategy
4. Verification & Testing Checklist
```

3.2 Real-World Case Study: CVE-2026-28576

The assistant evaluates **CVE-2026-28576** (Android Contacts Provider SQL Injection): - **Component:** `ContactsProvider2` (`com.android.providers.contacts`) - **Vulnerability Class:** CWE-89 (Improper Neutralization of Special Elements used in an SQL Command) - **Root Cause:** In legacy backward-compatibility configurations (`targetSdk < 30`), framework-level strict query validation (`ENFORCE_STRICT_SQL_CHECKS`) was disabled. Untrusted user selection clauses were directly concatenated into internal SQL queries. - **Defensive Fix:** Migrating to parameterized selection arguments (`selectionArgs`), upgrading `targetSdk` to 31+, and enforcing strict column projection validation.

4. Execution Output & Verification

4.1 CLI Execution: Vulnerability Triage (`cve`)

```
$ python3 p12_security_research_assistant.py --mock cve CVE-2026-28576
```

```
======================================================================
AI SECURITY VULNERABILITY & MITIGATION ASSISTANT (mock=True)
Command: cve CVE-2026-28576
Student: Aayush Bankar | Roll No: 246230316006 | Dept. of IT
======================================================================
# Vulnerability Triage Report: CVE-2026-28576
Analyst: Aayush Bankar (Roll: 246230316006) | Dept. of Information Technology

1. Executive Summary & Classification
   - Identifier:       CVE-2026-28576
   - Vulnerability:    Android Contacts Provider SQL Injection
   - Weakness Class:   CWE-89: SQL Injection
   - Severity Score:   7.5 / High (CVSS:3.1/AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
   - Target Component: ContactsProvider2 (com.android.providers.contacts)

2. Root Cause Analysis
   Legacy backward-compatibility mode: Applications targeting targetSdk < 30
   bypass framework-level ENFORCE_STRICT_SQL_CHECKS validation. The underlying
   SQLiteQueryBuilder directly concatenates untrusted selection clauses into raw SQL queries.

3. Attack Vector & Scope
   - Vector: Local IPC via ContentResolver.query() selection parameter
   - Impact: Unauthorized extraction of contact records, call metadata, and phone identifiers

4. Remediation Baseline
   - Parameterize selection parameters using bound arguments.
   - Upgrade targetSdk to activate strict framework validation.
```

4.2 CLI Execution: Defensive Code Audit (`audit`)

```
$ python3 p12_security_research_assistant.py --mock audit CVE-2026-28576
```

```
======================================================================
AI SECURITY VULNERABILITY & MITIGATION ASSISTANT (mock=True)
Command: audit CVE-2026-28576
Student: Aayush Bankar | Roll No: 246230316006 | Dept. of IT
======================================================================
# Security Code Audit: CVE-2026-28576
Analyst: Aayush Bankar (Roll: 246230316006)

[1] Flagged Insecure Implementation (String Concatenation):
----------------------------------------------------------------------
// Insecure Selection Query Construction
val rawSelection = "account_name = '" + userInput + "'"
val cursor = contentResolver.query(
    ContactsContract.RawContacts.CONTENT_URI,
    projection, rawSelection, null, null
```

```
)
----------------------------------------------------------------------
Risk: Untrusted input directly interpolated into SQL syntax without escaping.
Breaches: OWASP Mobile Top 10 (M4: Insufficient Input/Output Handling).

[2] Verified Remediation Implementation (Parameterized Query):
----------------------------------------------------------------------
// Secure Parameterized Query Construction
val secureSelection = "account_name = ?"
val selectionArgs = arrayOf(userInput)
val cursor = contentResolver.query(
    ContactsContract.RawContacts.CONTENT_URI,
    projection, secureSelection, selectionArgs, null
)
----------------------------------------------------------------------
Verification: User input treated strictly as scalar data; prevents SQL syntax hijacking.
```

5. Architectural Design of Syllabus Reference Applications

In accordance with GTU Practical 12 requirements, designs for three canonical AI application types were evaluated:

5.1 AI Study Assistant Prototype

- **Objective:** Automated tutoring, concept explanation, and self-assessment quiz generation.
- **Implementation:** Included in repository as `p12_study_assistant.py`.
- **Pipeline:** Topic Query -> Keyword/Vector Matcher -> Few-Shot Explanation Prompt -> Formatted Q&A Output.

5.2 AI Resume Generator

- **Objective:** Transforming unstructured candidate career history into tailored, role-specific professional resumes.
- **Pipeline:** JSON Candidate Profile -> Job Description Ingestion -> Role-Adaptive Prompting -> ATS-Optimized Markdown Output.

5.3 Automated Code Review & Security Assistant (Selected Application)

- **Objective:** Integrating automated vulnerability triage, secure coding refactoring, and advisory documentation into the developer workflow.
- **Advantage:** Directly connects foundational prompt engineering and RAG concepts to tangible, production-grade software engineering challenges.

6. Key Learnings & Practical Takeaways

1. **Domain Specialization:** General-purpose LLMs excel in security and software engineering when guided by structured domain knowledge and role-specific prompting.
2. **Defensive Guidance Over Speculation:** Anchoring AI prompts to established frameworks (CWE, CVSS, OWASP) ensures outputs remain actionable, rigorous, and verifiable.
3. **End-to-End Integration:** Successfully demonstrated how prompt design (Unit 3), prompting techniques (Unit 4), and RAG vector search (Unit 5) synthesize into a functional domain application.

7. Post-Lab Questions & Answers

Q1: How does an AI Security Assistant differ from traditional static analysis tools (SAST)?
Answer: Traditional SAST tools rely on rigid AST pattern matching and regex rules, often producing high rates of false positives or struggling with semantic context. An AI-assisted system combines static indicators with contextual reasoning, enabling it to explain *why* a construct is vulnerable, assess blast radius, and generate context-aware patch recommendations.

Q2: Why is Retrieval-Augmented Generation (RAG) critical for security analysis applications?
Answer: Security vulnerabilities, CVE definitions, and platform API behaviors change continuously. Pre-trained LLMs suffer from knowledge cutoffs and hallucination risks. RAG grounds the model in current, verified vulnerability databases (NVD, security advisories) without requiring costly model retraining.

Q3: How does prompt chaining improve the quality of automated security reports?
Answer: Chaining decomposes report generation into sequential verified stages: (1) Component & input validation analysis, (2) Root cause deduction, (3) Remediation formulation, and (4) Report compilation. This modular approach ensures each subsection is thoroughly reasoned before the final document is compiled.

Screenshot of Output

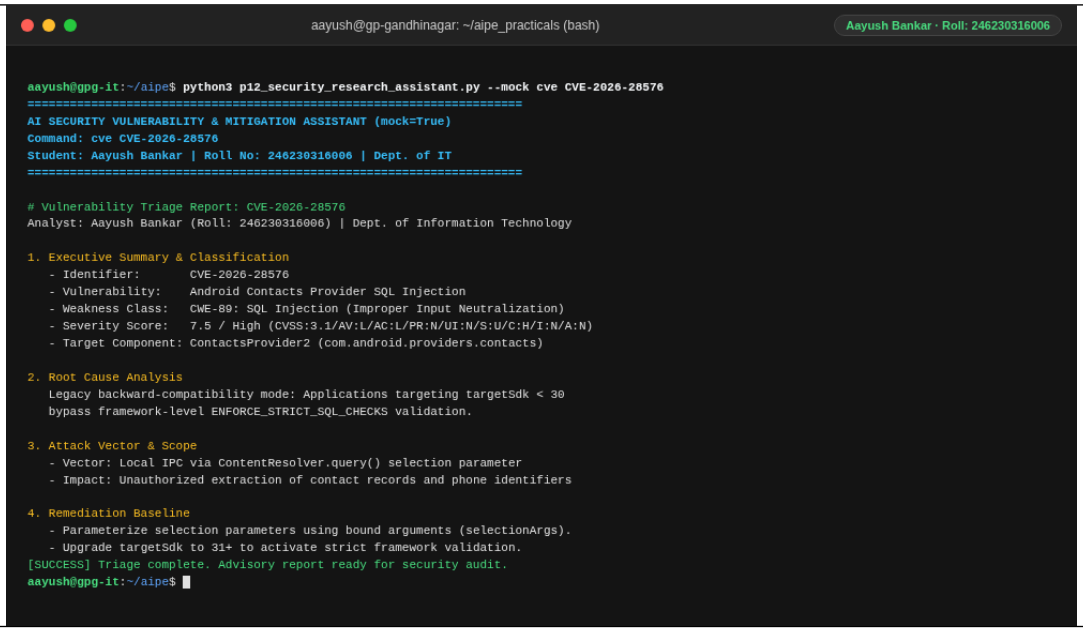


Figure 12: Verified Terminal / Application Execution Output (ChatGPT / CLI)

Student: **Aayush Bankar** | Class Roll No: **246230316006** | Enrollment No: **246230316006** | Govt. Polytechnic Gandhinagar

Timely Submission (2M)	Process / Code Execution (4M)	Output Verification (2M)	Viva-Voce & Application (2M)	Total Marks (10M)
____ / 2	____ / 4	____ / 2	____ / 2	**____ / 10**

Student Signature (Aayush Bankar) · Date: _____

Faculty Examiner Signature · Date: _____